Journal of Cybersecurity, 2026, pp. 1–20
DOI added during production
Published: Date added during production
Paper

Glaukopis: Building a 32B CTI LLM Using Structured Finetuning

Peter J. Worth Jr.,[1,*] Ionut Cardei[1] and Md Tanvirul Alam[1]

[1]Department of Electrical Engineering and Computer Science, Florida Atlantic University, 777 Glades Road, 33431, Boca Raton, FL, United States

[*]Corresponding author. pworth@athenasecuritygroup.com

Abstract

Cyber threat intelligence (CTI) workflows in regulated security operations cannot in general rely on frontier proprietary models due to cost, latency, and data-handling constraints, while generic foundation models lack the schema fidelity, applied reasoning, and current-threat awareness those workflows require. This paper introduces Glaukopis, a methodology and working system for producing on-premises 32B-scale CTI expert language models via structured finetuning—a variant of instruction finetuning in which all training data is computed by typed queries against an authoritative knowledge graph, and the supervised fine-tuning (SFT) pass is decomposed into a chain of stages with distinct datasets and hyperparameter regimes. Glaukopis has three components: Ariadne, a Neo4j substrate ingesting twelve public CTI sources (ATT&CK, ENGAGE, CAPEC, CWE, D3FEND, CVE 5.0, NVD, CISA KEV, FIRST EPSS, Sigma, ExploitDB, PoC-in-GitHub) into $\sim$280K nodes and $\sim$480K edges; Sophia, a graph-template generator that compiles declarative CTI reasoning patterns into Alpaca-format instruction triples whose answers are guaranteed consistent with the graph; and a four-stage chained SFT recipe (Core, +TAA, CSE, Recalibrate-32B). The v21 ship checkpoint reaches 66.3 average CTI benchmark score (vs. 59.0 for its Qwen-2.5-32B-Instruct base), approaching DeepSeek-V4-Pro (70.4), Gemini-3-Flash-Preview (69.8), and GPT-5.5 (72.9) at approximately two orders of magnitude lower per-evaluation cost.

Keywords structured finetuning; instruction fine-tuning; chained SFT; small language models; cyber threat intelligence; CTI knowledge graphs; Neo4j; Ariadne; Sophia; MITRE ATT&CK; AthenaBench; CyberSOCEval; CyberMetric

Abbreviations ATT&CK, CAPEC, CoT, CTI, CVE, CWE, EPSS, IFT, KEV, KG, LLM, MoE, NVD, SFT, SLM, SOC, TAA, TTP, VSP

Introduction

Large language models (LLMs) are now embedded throughout the cyber security workflow: they summarize threat reports, map indicators to MITRE ATT&CK, triage vulnerabilities, draft detection rules, and answer analyst questions inside security operations centers (SOCs). Surveys of LLMs for security span vulnerability detection, malware analysis, cyber threat intelligence (CTI), phishing detection, and SOC automation, and consistently identify the same residual failure modes: hallucinations on schema-bearing fields, shallow pattern matching where causal or multi-hop reasoning is required, and a structural inability to operate on threat information more recent than the base model's pretraining cutoff Levi et al. [2024], Liu et al. [2024].

The operational dilemma.

The practical response in industry has been an awkward two-way split. Generic foundation models are inexpensive and easy to host but underperform on CTI-specific tasks: schema fidelity (CVE/CWE/ATT&CK identifiers, CVSS vectors), applied reasoning (root-cause mapping, severity estimation, threat-actor attribution), and current-threat awareness are all weak. Frontier proprietary models—the largest GPT, Gemini, and DeepSeek releases—substantially close the capability gap, but their cost, latency, and data-handling characteristics are incompatible with most regulated security operations. SOCs supporting healthcare, financial services, or government workloads cannot, in general, send raw threat telemetry to a third-party API; even where the legal posture allows it, the per-evaluation cost of running a frontier model across a realistic CTI workload is prohibitive at the scale of a real analyst team.

Structured finetuning.

A natural alternative is to produce a small (sub-32B) language model that has been specifically post-trained on CTI material drawn from authoritative public sources, deployable on-premises behind organizational boundaries. The approach pursued in this paper, which we call *Structured Finetuning*, is a variant of instruction finetuning (IFT) Wei et al. [2022a], Ouyang et al. [2022], Chung et al. [2022] that adds two further constraints: (i) every training row is computed by a typed query against an authoritative knowledge graph, so each Alpaca-format (instruction, input, output) triple is grounded in named graph nodes and edges rather than synthesized from free-form text; and (ii) the SFT pass is decomposed into a chain of stages with distinct dataset shards and hyperparameter regimes (Section 2.6), so that capabilities accumulate compositionally and

Received: Date Month Year. **Revised received:** Date Month Year. **Accepted:** Date Month Year

per-axis regressions are attributable to specific stages. Structured finetuning is thus not a new training paradigm but a particular discipline imposed on IFT data generation and SFT scheduling; its closest neighbors in the literature are the CyberPal.AI / SecKnowledge line, which couples expert-curated instruction schemas with chain-of-thought enrichment Levi et al. [2024, 2025], and the SEvenLLM and CyberBench efforts, which adapt smaller open models to security through hand-crafted or LLM-synthesized instruction corpora Ji et al. [2024], Liu et al. [2024]. Section 2.7 traces the three research traditions that converge in structured finetuning—schema-constrained generation, knowledge-graph-grounded QA, and expert-schema-driven IFT—and locates Glaukopis within them.

The Glaukopis system.

Glaukopis is the working instantiation of structured finetuning described in this paper. It has three named components, each developed in detail in a later section:

- **Ariadne** (Section 4), a Neo4j knowledge-graph substrate that ingests twelve public CTI sources spanning offensive technique catalogues (ATT&CK, ENGAGE), weakness and attack-pattern taxonomies (CWE, CAPEC), defensive countermeasures (D3FEND), vulnerability records (CVE 5.0, NVD), exploitation signals (CISA KEV, FIRST EPSS), detection content (Sigma), and weaponized-exploit evidence (ExploitDB, PoC-in-GitHub) into approximately 280,000 nodes and 480,000 edges, refreshed daily for current-threat coverage.
- **Sophia** (Section 4.4), a graph-template generator that compiles declarative CTI reasoning patterns into Alpaca-format instruction triples whose answers are computed directly from Cypher queries against Ariadne. Sophia supports positive and negative examples, cross-schema reasoning chains (e.g., CVE→CWE→CAPEC→ATT&CK), and incremental regeneration aligned to specific time windows—a property that lets the training corpus track the threat landscape as it evolves.
- **A four-stage chained SFT recipe** (Section 5.4) that turns Sophia-generated triples into a sequence of cumulative checkpoints—Core, +TAA, CSE, and Recalibrate-32B—each with its own dataset shard, hyperparameter regime, and intended capability outcome. Chaining the stages, rather than mixing all training data into one pass, is what makes per-axis ablation, incremental refresh, and known-trade-off recovery tractable in this pipeline. The principle is reviewed in Section 2.6 below.

Headline result.

The ship checkpoint produced by the v21 vintage of this pipeline, `asg-ai/athena-cti-sft-qwen25-32b-v21-recal-32b`, reaches an average CTI-benchmark score of 66.3 across AthenaBench, CyberSOCEval, and CyberMetric. The same architecture's base instruct model (`Qwen/Qwen2.5-32B-Instruct`) scores 59.0 on the same suite—so the structured-finetuning chain adds 7.3 points without changing the parameter count or runtime footprint. Against frontier reference points, the ship checkpoint sits within a few points of DeepSeek-V4-Pro (70.4), Gemini-3-Flash-Preview (69.8), and GPT-5.5 (72.9), at a small fraction of their evaluation cost. Detailed numbers are reported in Section 6.

Paper structure.

The remainder of the paper is organized as follows:

- Section 2 reviews instruction finetuning, supervised fine-tuning, the chained-SFT refinement, the structured-finetuning variant we adopt, and the cyber-security and SLM literature that the present work builds on.
- Section 3 surveys cyber security LLM benchmarks and develops the four-suite evaluation portfolio (AthenaBench, CyberSOCEval, CyberMetric, MMLU-Pro) used in this paper.
- Section 4 introduces Ariadne—the Athena CTI knowledge-graph substrate—and Sophia, the graph-template generator that compiles CTI reasoning patterns into instruction triples consumed by the Glaukopis training pipeline.
- Section 5 describes Glaukopis itself: the v21 template-generation vintage, the four-stage chained SFT recipe, and the evaluation harness.
- Section 6 presents the v21 evaluation results across the benchmark portfolio.
- Section 7 closes with open challenges and research directions.

Instruction Fine-Tuning, SFT, and Structured Finetuning

This section reviews the research foundations on which Glaukopis stands. We begin with the pretraining-to-instruction transition and the original IFT pipeline (§2.1), then survey the three load-bearing IFT papers—FLAN, InstructGPT, and FLAN-T5—and what they jointly imply for domain-specialized instruction tuning (§§2.2–2.5). We then describe two refinements that shape the Glaukopis recipe specifically: *chained SFT*, the practice of composing multiple SFT stages with distinct datasets and hyperparameter regimes (§2.6); and *structured finetuning*, the variant of IFT we adopt in which all training data is computed by typed queries against an authoritative knowledge graph (§2.7). We close with two cyber-security–specific subsections that situate the present work within the CyberPal.AI / SecKnowledge line and the broader Small Language Model (SLM) literature (§§2.8–2.9).

From Pretraining to Instruction-Following

Pretrained LLMs are optimized for next-token prediction on broad text corpora, not for following explicit user instructions. Instruction tuning—also called instruction fine-tuning (IFT)—fine-tunes such models on labeled datasets of instructions and target outputs, improving their ability to interpret prompts as tasks rather than mere text prefixes Wei et al. [2022a], Ouyang et al. [2022].

Key characteristics of instruction tuning include:

- **Data format.** Each sample consists of an instruction, optional context, and a desired output.
- **Supervised objective.** The model is trained to maximize log-likelihood of the reference output given the instruction and context, often combining a standard language modeling loss with an instruction-following loss.
- **Generalization across tasks.** Instruction datasets deliberately mix many task types (QA, classification, summarization, transformation) so that the fine-tuned model can generalize to novel instructions, as shown by InstructGPT and FLAN Wei et al. [2022a], Ouyang et al. [2022].

IFT is now routine in the model lifecycle: base models (e.g., LLaMA-like, Granite, Qwen) are first pretrained, then instruction-tuned, and often further refined via reinforcement learning from human feedback (RLHF) and safety fine-tunes Ouyang et al. [2022].

The modern formulation of instruction fine-tuning coalesced through three seminal research efforts between 2021 and 2022. Together, they established the empirical phenomenon, the alignment pipeline, and the scalability properties that underpin nearly all contemporary instruction-tuned models, including domain-specialized systems such as CyberPal.AI and the Athena CTI instruction framework. We review each in turn.

FLAN: Instruction Tuning as a Generalization Paradigm

The first widely recognized formulation of instruction fine-tuning was introduced by Wei et al. in *Finetuned Language Models Are Zero-Shot Learners* (FLAN) Wei et al. [2022a]. This work demonstrated that large pretrained language models, when fine-tuned on a sufficiently diverse collection of tasks expressed uniformly as natural-language instructions, exhibit strong zero-shot generalization to unseen tasks.

FLAN unified dozens of existing NLP datasets by converting each task into multiple instruction templates (e.g., question answering, sentiment analysis, textual entailment, translation). The authors showed that instruction formatting itself—explicitly describing the task in natural language—was a critical factor in enabling generalization, often rivaling or surpassing few-shot prompting on much larger models.

The central insight of FLAN is that *instruction-following is a learnable capability* rather than an emergent property of scale alone. By training on heterogeneous instruction–response pairs, the model internalizes a latent task distribution that allows it to interpret novel instructions without task-specific fine-tuning. This result established instruction fine-tuning as a distinct and powerful paradigm, forming the conceptual foundation for subsequent work in alignment and domain adaptation.

InstructGPT: Instruction Following and Alignment with Human Feedback

While FLAN established the effectiveness of instruction fine-tuning for generalization, it did not directly address issues of safety, helpfulness, or alignment with human preferences. These concerns were tackled by Ouyang et al. in *Training Language Models to Follow Instructions with Human Feedback* (InstructGPT) Ouyang et al. [2022].

InstructGPT introduced a three-stage training pipeline that remains the basis for many production LLMs:

1. **Supervised instruction fine-tuning (SFT)** on human-written instruction–response pairs;
2. **Preference modeling** using human rankings of multiple model outputs;
3. **Reinforcement learning from human feedback (RLHF)** to optimize model behavior toward preferred responses.

A key result of this work was that relatively small models, when instruction-tuned and aligned via RLHF, were preferred by human evaluators over much larger pretrained models. This demonstrated that alignment and instruction-following quality can outweigh raw model scale.

From a historical perspective, InstructGPT complements FLAN by transforming instruction fine-tuning from a purely performance-driven technique into a broader alignment framework. Together, FLAN and InstructGPT established instruction fine-tuning as both a generalization mechanism and a control interface between humans and large language models.

FLAN-T5: Scaling Instruction Fine-Tuning

The third pillar of instruction fine-tuning was established by Chung et al. in *Scaling Instruction-Finetuned Language Models* (FLAN-T5) Chung et al. [2022]. This work systematically explored how instruction fine-tuning behaves across model sizes, task mixtures, and dataset scales.

FLAN-T5 expanded the instruction corpus to thousands of tasks and demonstrated that instruction fine-tuning consistently improves performance across a wide range of downstream benchmarks, including reasoning and knowledge-intensive tasks. Importantly, the authors showed that instruction fine-tuning benefits models of all sizes, from small to very large, and that gains persist even as base models scale.

This result established instruction fine-tuning as a *scalable and robust training paradigm*, rather than a technique limited to a narrow regime. It also reinforced the idea that the diversity and structure of instruction data are as important as parameter count—a conclusion that directly motivates modern domain-specialized instruction datasets.

Implications for Domain-Specialized Instruction Tuning

Together, FLAN, InstructGPT, and FLAN-T5 define the historical and conceptual foundations of instruction fine-tuning:

- FLAN establishes that instruction-following can be learned and generalized;
- InstructGPT integrates instruction fine-tuning with human alignment objectives;
- FLAN-T5 demonstrates that instruction fine-tuning scales across model sizes and task mixtures.

Recent domain-specific efforts—such as CyberPal.AI and SecKnowledge Levi et al. [2024, 2025]—extend these principles by grounding instruction data in structured cyber security knowledge bases. Two refinements of the basic IFT pipeline are particularly relevant to the present work, both reviewed next: *chaining*, the practice of decomposing the SFT pass into multiple stages with distinct datasets and hyperparameter regimes; and *structured finetuning*, the practice of computing every training row by a typed query against an authoritative knowledge graph rather than synthesizing it from free-form text.

Chaining SFT: Multi-Stage Supervised Fine-Tuning

A recurring design choice in modern post-training is to compose several supervised fine-tuning (SFT) stages into a chain rather than to perform a single, monolithic instruction tune. In a chained SFT pipeline, the base pretrained model is adapted to its target deployment through an ordered sequence of stages, each with its own dataset, optimization hyperparameters, and intended outcome; the output checkpoint of each stage becomes the initialization for the next. Capabilities are thereby accumulated compositionally instead

of competing for capacity within a single fine-tune, and stages can be re-run independently when their underlying data changes.

The staged structure of modern LLM post-training is by now well established. InstructGPT formalized a three-phase pipeline—supervised instruction fine-tuning, preference modeling, and reinforcement learning from human feedback—in which each phase has its own data and objective and the order of the stages is load-bearing for the final model's behavior Ouyang et al. [2022]. FLAN and FLAN-T5 showed earlier that instruction tuning is most effective when it follows a well-pretrained base rather than replacing pretraining, that the diversity and structure of instruction data matter at least as much as parameter count, and that the gains of instruction tuning persist across model scales Wei et al. [2022a], Chung et al. [2022]. The domain-adaptive pretraining literature, notably the "Don't Stop Pretraining" line, demonstrated that continued training of a general model on in-domain text yields measurable downstream improvements and that the order and composition of training stages materially affect final task performance Gururangan et al. [2020].

Several practical motivations recur across these results. First, separating broad domain absorption from precise task behavior reduces destructive interference: training jointly on noisy domain text and structured task triples tends to either underfit the structured task or overfit narrow templates, while staging gives each objective its own learning-rate, schedule, and packing regime Gururangan et al. [2020], Ouyang et al. [2022]. Second, reasoning supervision is more effective when it is layered on top of a model that already commands the underlying domain vocabulary and task formats than when it is mixed in from the start; this is the design that motivated the chain-of-thought enrichment stages in CyberPal/SecKnowledge 2.0 and analogous reasoning-focused fine-tunes Wei et al. [2022b], Levi et al. [2025]. Third, staging makes pipelines incrementally refreshable: when only one data source updates, only the stages that consume it need to be re-run, and intermediate checkpoints can be evaluated to attribute downstream gains to specific stages.

In the cyber security setting in particular, the CyberPal.AI / SecKnowledge line operationalizes this pattern explicitly, separating expert instruction-schema construction, knowledge-grounded answer synthesis, and chain-of-thought rationale enrichment into distinct dataset-construction and training stages, with measurable benefits from the staged structure Levi et al. [2024, 2025]. Glaukopis, described in Section 5, adopts the same general philosophy with a four-stage chain tailored to graph-template-generated CTI instruction data and the AthenaBench / CyberSOCEval / CyberMetric evaluation portfolio.

Structured Finetuning: History and Context within IFT

The second refinement specific to Glaukopis is *structured finetuning*: a variant of IFT in which every training row is computed by a typed query against an authoritative knowledge graph (or other formally specified data source) rather than authored by hand or synthesized by an LLM. Structured finetuning is not a single canonical research lineage in the literature the way FLAN-style IFT or InstructGPT-style alignment are; rather, it sits at the intersection of three research traditions that have been converging over the past decade, and Glaukopis is one concrete instantiation of that convergence.

Tradition 1: Schema- and grammar-constrained generation.

The earliest precursor is the semantic parsing and constrained-decoding literature, in which target outputs are required to conform to a formal grammar or schema (logical forms, SQL queries, knowledge-base triples). Training data in this line is computed rather than written: pairs of natural-language inputs and structured outputs are generated by sampling the formal grammar or by executing queries against a reference database. The resulting models learn to produce outputs that are not just plausible text but valid instances of a typed object. This line is the conceptual origin of the idea that "every training row should be computable, not authored."

Tradition 2: Knowledge-graph-grounded QA and KG-to-text.

A second tradition—knowledge-graph-grounded question answering and KG-to-text generation—contributes the specific pattern of *materializing training rows by graph traversal*. A typed query against a knowledge graph returns a subgraph; that subgraph is rendered, by a template or a learned model, into a natural-language question–answer pair that the LLM is then trained on. The CTI-specific literature reviewed in Section 4.5 (Fieblinger et al. Fieblinger et al. [2024], Open-CyKG Wu et al. [2022], CTIKG Yu et al. [2024], K-CTIAA Zhao et al. [2024], and the broader cyber knowledge-graph effort Zhang et al. [2023]) is the cyber-domain instantiation of this pattern. What these efforts collectively establish is that LLMs can be trained against graph-derived QA corpora with measurable benefits on tasks where the underlying graph is the authoritative source of truth.

Tradition 3: Expert-schema-driven IFT for cyber security.

The third tradition is the line most immediately adjacent to Glaukopis: IBM's CyberPal.AI Levi et al. [2024] and the SecKnowledge 2.0 follow-on Levi et al. [2025]. These works show that an instruction corpus structured by expert-authored schemas—"map this log entry to an ATT&CK technique," "suggest a mitigation for this CVE"—can transform a general SLM into a cyber security expert at the 4–20B scale. CyberPal does not require the answers to be materialized from a knowledge graph (their schemas are populated semi-automatically from a mix of structured and unstructured sources), but the discipline of constraining the instruction shape via expert schemas is conceptually neighboring.

What Glaukopis adds.

The Glaukopis instantiation of structured finetuning combines all three traditions in a specific way: (i) the schema is the typed Ariadne graph schema (Tradition 1); (ii) every training row is materialized by a Cypher query against the populated Ariadne graph (Tradition 2); and (iii) the templates that drive those queries are CTI-specific reasoning patterns authored or curated by domain experts in the CyberPal sense (Tradition 3). Section 5.1 describes the Sophia generator that implements this combination. The point of using the term *structured finetuning* for this composition—rather than "IFT," "KG-grounded fine-tuning," or "expert-schema fine-tuning" individually—is to highlight that the discipline is the union of all three: typed schema, graph materialization, and expert-authored reasoning patterns, with chained-SFT scheduling (Section 2.6) as the orthogonal training-side complement. As a working term it captures

what each of the constituent traditions on its own does not: a complete pipeline in which schema, data materialization, training rows, and SFT stage decomposition are all explicit, typed, and auditable.

Instruction Fine-Tuning for Cyber Security

Cyber Security introduces domain-specific challenges:

- Technical jargon and heterogeneous schemas (CVE, CWE, CAPEC, ATT&CK).
- Need for structured, evidence-grounded outputs (e.g., CVSS vectors, CWE IDs, ATT&CK mappings).
- Rapid concept drift as new vulnerabilities and campaigns appear.

Several efforts address this via cyber-specific instruction data:

- **SEvenLLM.** Ji et al. construct a bilingual cyber security instruction dataset and a CTI benchmark (SEvenLLM-Bench) from real incident reports, converting raw text into supervised QA, extraction, and reasoning tasks across many task types Ji et al. [2024].
- **CyberBench / CyberInstruct.** Liu et al. create CyberInstruct, a set of cyber-specific instruction tasks drawn from logs, alerts, and vulnerability descriptions, used to train models evaluated on CyberBench Liu et al. [2024].
- **Lily-Cybersecurity-7B.** Lily is a Mistral-7B fine-tune on hand-crafted cyber security and hacking instructions, later enriched by an LLM for style and consistency Liu et al. [2024].

The **CyberPal.AI** line of work from IBM pushes this further by tightly integrating CTI knowledge bases and detection artifacts into an expert-driven instruction pipeline:

- **SecKnowledge (CyberPal 1.0).** Levi et al. build an instruction dataset by mining MITRE ATT&CK, Sigma rules, SIEM rules, adversary emulation plans, and threat reports, then using domain experts to define instruction schemas (e.g., mapping logs to ATT&CK techniques, suggesting mitigations) and populate them semi-automatically Levi et al. [2024].
- **IFT results.** CyberPal models instruction-tuned on SecKnowledge (based on LLaMA-like and other open models) outperform base models by up to roughly 24% on security tasks and show improved robustness on CTI benchmarks Levi et al. [2024].

The follow-on **"Toward Cybersecurity-Expert Small Language Models"** paper extends this to SecKnowledge 2.0, enriching instructions with chain-of-thought (CoT) rationales and designing a pipeline with expert-in-the-loop validation and LLM-guided knowledge retrieval, then training cyber security expert SLMs (4B–20B parameter models) that rival or surpass much larger general models on CTI tasks Levi et al. [2025].

These works demonstrate that expert-designed, knowledge-grounded instruction datasets can transform general LLMs into specialized CTI assistants with strong reasoning and reduced hallucination—precisely the niche targeted by Glaukopis's structured finetuning pipeline, with the added discipline that every training row is materialized from the Ariadne CTI graph rather than synthesized.

Small Language Models for Cyber Security

SLMs—roughly 2–20B parameters, with 32B at the natural upper bound for on-premises deployment—are increasingly attractive for cyber security:

- **Deployment constraints.** Many SOC and CTI environments require on-premises, air-gapped, or "bring-your-own-model" setups for regulatory and confidentiality reasons.
- **Latency and cost.** Incident response and detection engineering often require low-latency responses and large volumes of queries; SLMs are simpler to host and scale.
- **Specialization.** With strong domain IFT, SLMs can match or exceed larger general models on specialized tasks while being easier to retrain as CTI sources evolve Levi et al. [2025].

CyberPal 2.0 explicitly targets this: it compares cyber security expert SLMs (4B–20B) against both larger open models and proprietary systems across CTI knowledge tasks, finding that well-instruction-tuned SLMs can achieve competitive or superior performance on several domains, especially when trained with CoT-enriched instructions and CTI-aware formatting Levi et al. [2025].

The emerging picture is that *structured finetuning + chained SFT + SLM* is a pragmatic recipe for CTI: the heavy lifting (linguistic knowledge) comes from pretraining; domain semantics and task formats come from graph-materialized IFT; capability accumulation comes from chaining; and runtime and deployment practicality come from SLM size. The Glaukopis instantiation of this recipe—including the Ariadne graph substrate, the Sophia template generator, the four-stage SFT chain, and the evaluation harness—is described in Section 5.

Cyber Security LLM Benchmarks and AthenaBench

Why Cyber-Security-Specific Benchmarks?

General benchmarks such as GLUE, MMLU, and BIG-Bench measure broad language understanding and world knowledge, and certain security-adjacent capabilities (e.g., information-security multiple-choice questions in MMLU) appear as minor subsets, but these suites are not designed to probe the workflows that actually define security practice. They lack the structured schemas, evidence-grounded reasoning, and threat-current content that distinguish cyber security tasks from general knowledge work. The gaps that motivate domain-specific benchmarking are concrete:

- **Schema fidelity.** Security workflows operate over standardized identifiers and structured artifacts (CVE, CWE, CAPEC, CVSS vectors, MITRE ATT&CK technique IDs, STIX/TAXII objects). General benchmarks do not test whether a model produces well-formed outputs in these schemas, which is the difference between a usable assistant and one that requires constant human re-formatting.
- **Applied reasoning.** CTI and SOC analysts rarely need pure recall; they need chained inferences (e.g., from a CVE description to the underlying weakness class, then to relevant attack patterns and detection opportunities), severity estimation under partial information, and abductive attribution from observed TTPs. These reasoning shapes are largely absent from general benchmarks.

- **Temporal relevance.** The threat landscape evolves on a weekly cadence (new CVEs, new APT activity, new exploitation campaigns), so static benchmarks rapidly drift from operational reality and become measures of stale memorization rather than current competence Alam et al. [2025].
- **Data contamination.** Many cyber security sources (MITRE ATT&CK, the NVD, public threat reports) are heavily represented in LLM pretraining corpora. A static benchmark drawn from those same sources risks measuring recall of training data rather than genuine reasoning, an effect that dynamic benchmarks aim to mitigate Alam et al. [2025].

In response, the past two years have seen a rapid expansion of cyber-security-specific benchmarks, which can be grouped roughly into four overlapping families: *knowledge benchmarks*, focused on factual recall of security concepts and terminology; *secure-coding and offensive-capability benchmarks*, focused on whether models help or hinder secure software development and adversarial use; *CTI and SOC benchmarks*, focused on applied threat-intelligence and operations reasoning; and *dynamic / live-source benchmarks*, focused on resistance to data staleness and contamination. Important entries include:

- **CyberMetric.** A retrieval-augmented, certification-style multiple-choice benchmark by Tihanyi et al. covering nine cyber security domains, released in graduated sizes (80, 500, 2000, and 10 000 questions) to support both rapid evaluation and statistically robust comparison Tihanyi et al. [2024].
- **SecEval.** A multiple-choice benchmark of cyber security knowledge questions spanning multiple security subdomains, used to evaluate factual coverage of foundation models on standard certification-style content Hu et al. [2024].
- **CyberBench.** A multi-task benchmark for cyber security LLMs that includes classification, question answering, and log/report analysis, accompanied by the CyberInstruct training corpus used by the same authors to build cyber-tuned models Liu et al. [2024].
- **CTIBench.** A dedicated CTI benchmark covering CTI knowledge, root-cause mapping, vulnerability severity prediction, threat-actor attribution, threat-knowledge graph construction, and attack-technique extraction; constructed from curated CTI corpora and designed for reasoning-intensive evaluation Alam et al. [2024].
- **SEvenLLM-Bench.** A bilingual CTI benchmark derived from real incident reports, designed alongside the SEvenLLM-Instruct dataset and emphasizing cross-lingual security comprehension Ji et al. [2024].
- **Purple Llama CyberSecEval and CyberSecEval 2.** Benchmarks from Meta evaluating secure-code generation, susceptibility to assisting with cyberattacks, prompt-injection robustness, and related risk dimensions Bhatt et al. [2023, 2024].
- **WMDP.** The Weapons of Mass Destruction Proxy benchmark by Li et al. includes a cyber security subset measuring potentially hazardous cyber-offense knowledge and is widely used to study unlearning and safety-capability tradeoffs in security-adjacent capabilities Li et al. [2024].
- **AthenaBench.** A dynamic CTI benchmark that draws from live sources (MITRE ATT&CK, NVD, CISA advisories, threat reports) and explicitly targets the staleness and contamination problems that limit static suites; discussed in detail below Alam et al. [2025].

Together, these benchmarks shift evaluation from generic question answering toward security-aware reasoning, but they cover very different surfaces—factual recall, secure-code generation, applied CTI reasoning, and dynamic resistance to drift—and no single benchmark is sufficient on its own. Glaukopis is therefore evaluated against a portfolio of complementary suites, with particular weight on CTIBench and AthenaBench (which directly target CTI reasoning), CyberMetric at the 2 000- and 10 000-question tiers (which provides statistically robust factual coverage), and CyberSOCEval's malware and threat-intelligence test suites (which probe operationally relevant SOC reasoning). The remainder of this section describes these benchmarks in more detail.

CyberMetric

CyberMetric is a multiple-choice cyber security knowledge benchmark constructed by Tihanyi et al. using a retrieval-augmented generation (RAG) pipeline over an authoritative corpus of cyber security standards, frameworks, NIST publications, and academic and practitioner references Tihanyi et al. [2024]. Candidate questions are generated by an LLM conditioned on retrieved source material, then filtered and validated by human cyber security experts before inclusion in the released dataset. The result is a benchmark whose questions are grounded in citable source material rather than constructed from scratch, which mitigates the common problem of LLM-as-author benchmarks whose answer keys reflect the generating model's biases more than the underlying domain.

CyberMetric covers nine cyber security domains: network security, cryptography, identity and access management, security policies and governance, threats and vulnerabilities, security operations, application and system security, cloud security, and incident response. It is released in four cumulative sizes—80, 500, 2 000, and 10 000 questions—designed to support different evaluation regimes: the 80- and 500-question tiers for rapid iteration and ablation, and the 2 000- and 10 000-question tiers for statistically robust comparison between models where small accuracy differences must be distinguished from sampling noise Tihanyi et al. [2024].

In Glaukopis, CyberMetric is used at the 2 000- and 10 000-question tiers. These tiers were chosen for three reasons. First, sample size: at 10 000 questions the standard error on accuracy estimates is approximately 0.5 percentage points, which is fine-grained enough to detect the kind of single-digit improvements typically produced by an additional SFT stage in the chained recipe (Section 5.4). Second, domain coverage: the larger tiers include enough questions per domain to support per-domain breakdowns, which lets us identify whether a given training-stage change improves balanced cyber security knowledge or only narrow subdomains. Third, comparability: published evaluations of contemporary models—both general-purpose frontier models and cyber-specialized SLMs—report CyberMetric scores at the 2 000- and 10 000-question tiers, allowing direct comparison without re-running baselines.

CyberMetric's principal limitation, shared with all multiple-choice benchmarks, is that it measures recognition rather than generation: a model can score well on CyberMetric without being able to produce structured CTI outputs or coherent multi-step reasoning. For this reason it is used in Glaukopis as one component of a broader evaluation portfolio rather than as a primary metric, and

is paired with reasoning-oriented suites (CTIBench, AthenaBench) and operationally focused suites (CyberSOCEval) that exercise generation and applied reasoning.

CyberSOCEval

CyberSOCEval is an evaluation suite oriented around security-operations-center (SOC) tasks, providing test suites that target operationally meaningful reasoning rather than pure knowledge recall Meta AI / Purple Llama Team [2025]. Whereas CyberMetric measures what a model knows about cyber security as a body of knowledge, and CTIBench/AthenaBench measure what a model can reason over a CTI knowledge graph, CyberSOCEval measures whether a model can perform the kind of analytic work that occupies an analyst inside a SOC: classifying observed artifacts, correlating evidence, and producing actionable summaries.

In Glaukopis, CyberSOCEval is used along two of its test suites:

- **Malware analysis suite.** Tasks in this suite present malware-related artifacts (samples, behavioral indicators, or analyst-style descriptions) and probe whether the model can extract family/variant attribution, characterize behavior in a structured way (e.g., what techniques the artifact exhibits), and identify defensive implications. This suite is operationally important because malware-related reasoning is one of the most frequent generation tasks asked of CTI assistants and is also one of the most failure-prone: errors here typically take the form of confident misattribution rather than refusal, which is more harmful than abstention.
- **Threat-intelligence suite.** Tasks here exercise the analyst-facing CTI reasoning chain: ingesting raw report or alert content, extracting TTPs and indicators, mapping them to standardized schemas (ATT&CK techniques, mitigations, threat-actor designations where supported by evidence), and producing structured CTI summaries. This complements AthenaBench's task structure but emphasizes free-form narrative input over knowledge-graph-derived queries, which makes it a useful test of how well a CTI SLM generalizes from training data structured by the Athena template generator to less-structured real-world inputs.

These two suites are particularly relevant to Glaukopis because they exercise capabilities that the chained-SFT pipeline (Section 5.4) is explicitly designed to develop. Stage 1's Phase B AthenaBench-catalog drill and Stage 2's TAA narrow drill train the model on schema-faithful answers to graph-derived questions, and Stage 3 (the CyberSOCEval letter-set drill) is itself a targeted preparation against this benchmark family; the malware and threat-intelligence suites then test whether the capabilities cultivated through these stages transfer to operationally realistic narrative inputs. Their inclusion alongside CyberMetric (factual coverage), CTIBench (CTI reasoning), and AthenaBench (dynamic CTI reasoning) gives the evaluation portfolio coverage across the four benchmark families identified above: knowledge, secure-coding/offensive capability via Purple Llama suites as needed, applied CTI reasoning, and operational SOC reasoning Tihanyi et al. [2024], Alam et al. [2024, 2025], Bhatt et al. [2023, 2024], Meta AI / Purple Llama Team [2025].

AthenaBench: A Dynamic CTI Benchmark

AthenaBench is introduced as a dynamic benchmark suite for evaluating LLMs on realistic, knowledge-intensive CTI tasks and explicitly extends CTIBench Alam et al. [2025, 2024]. Its key innovations include:

- **Dynamic data construction.** Instead of static corpora, AthenaBench connects to live CTI data sources and APIs (MITRE ATT&CK, NVD, CISA advisories, threat reports) to continuously generate benchmark samples within configurable time windows. Tasks such as root cause mapping (CVE→CWE) and severity prediction (CVSS) are drawn from recent CVEs and weaknesses, mitigating training-set leakage and static knowledge effects Alam et al. [2025].
- **Six complementary tasks.** AthenaBench defines: CTI Knowledge Test (CKT), Root Cause Mapping (RCM), Vulnerability Severity Prediction (VSP), Threat Actor Attribution (TAA), Risk Mitigation Strategy (RMS), and Attack Technique Extraction (ATE). Together, they probe CTI knowledge, root cause reasoning, structured severity prediction, abductive attribution, defensive planning, and TTP extraction Alam et al. [2025].
- **Enhanced metrics and aggregated score.** AthenaBench improves task-specific metrics and defines a unified aggregated score, making it easier to compare LLMs' CTI reasoning capabilities along different dimensions Alam et al. [2025].
- **Model evaluation.** The authors evaluate 12 LLMs, including GPT-4, GPT-4o, GPT-5, Gemini-2.5 Flash/Pro, Qwen variants, Llama 3/3.1/3.3, and a Llama-Primus merged model. Proprietary models (GPT-5, Gemini-2.5 Pro) lead overall, but all models struggle on reasoning-intensive tasks such as TAA and RMS, especially open-source ones Alam et al. [2025].

By using live CTI sources and emphasizing reasoning over current threats, AthenaBench moves CTI benchmarking closer to real-world analyst workloads and provides a stringent target for cyber security expert SLMs.

Relationship to the Athena Training Pipeline

AthenaBench and Glaukopis (powered by the Sophia graph-template generator over the Ariadne CTI substrate, both detailed in Section 4) are complementary components of a full CTI LLM lifecycle:

- **Training side (Sophia / Ariadne).** Sophia compiles CTI reasoning patterns into Cypher queries over the Ariadne graph (ATT&CK, CVE, CAPEC, CWE, ENGAGE, D3FEND, etc.) and emits instruction triples that drill CTI knowledge, multi-hop reasoning, negative cases, and structured outputs Cardei [2025].
- **Evaluation side (AthenaBench).** Dynamically constructs CTI tasks from the same or similar underlying sources (ATT&CK, NVD, CISA, threat reports) but with different pipelines and sampling strategies, providing independent, evolving test sets to evaluate model generalization Alam et al. [2025].

Many of Sophia's template patterns directly mirror AthenaBench tasks:

- CVE→CWE reasoning templates align with RCM.
- Templates that traverse CVE→CWE→CAPEC→ATT&CK map onto knowledge and mitigation reasoning tested in CKT and RMS.

- Templates involving malware, techniques, platforms, and data components parallel the kind of "who uses what and how is it detected" reasoning central to CTI analysis and attack technique extraction tasks Cardei [2025].

Sophia's incremental generation features (version and timestamp filtering) make it possible to align the training data's recency with AthenaBench's dynamic evaluation horizon, mitigating data leakage while ensuring models are trained on approximately the same vintage of CTI as they are evaluated on Cardei [2025], Alam et al. [2025].

This alignment positions the Sophia + Ariadne pipeline as a natural training counterpart to AthenaBench's evaluation side, much as SecKnowledge and CyberPal models are paired with CTI benchmarks in IBM's work Levi et al. [2024, 2025]. The next section describes the data substrate that makes this alignment concrete: Ariadne, the populated Neo4j CTI graph from which every Glaukopis training row is materialized.

Ariadne: A Knowledge Graph for CTI SFT

CTI Concepts and Workflows

Cyber Threat Intelligence (CTI) is commonly defined as the collection, analysis, and dissemination of information about threats and adversaries to inform defensive decisions McMillan [2013]. CTI workflows include:

- Ingesting unstructured reports (vendor blogs, whitepapers, advisories).
- Structuring them into entities and relations: vulnerabilities (CVEs), weaknesses (CWEs), attack patterns (CAPEC, ATT&CK), threat actors, campaigns, malware, tools, TTPs.
- Reasoning over this structure to answer questions such as:
  - Which techniques does this malware implement?
  - Which mitigations cover these techniques on a given platform?
  - Which threat actor is most likely behind this cluster of activity?
  - How severe and exploitable is a new CVE in our environment?

CTI sharing and CTI knowledge graphs emphasize the importance of standardized schemas (e.g., STIX/TAXII), community knowledge bases (MITRE ATT&CK, CAPEC, CWE), and APIs like NVD for building structured representations that can be consumed by automation Strom et al. [2018], mit [b,c,a].

MITRE Ecosystem and Related Schemas

The MITRE ecosystem underpins much of modern CTI:

- **MITRE ATT&CK.** A globally accessible knowledge base of adversary tactics, techniques, and sub-techniques, originally released in 2013 and formally documented in "MITRE ATT&CK: Design and Philosophy" Strom et al. [2018]. ATT&CK represents adversary behavior as a graph where attack-pattern nodes (techniques) relate to software, groups (threat actors), data sources, and mitigations.
- **CVE and NVD.** The Common Vulnerabilities and Exposures system and NIST's National Vulnerability Database provide machine-readable vulnerability records with descriptions, affected products, CVSS metrics, and links to related CWE and CAPEC entries mit [b].
- **CWE.** Common Weakness Enumeration captures abstract classes of software/hardware weaknesses, often linked from CVEs and CAPEC attack patterns mit [c].
- **CAPEC.** A catalog of attack patterns describing how adversaries exploit weaknesses and how those attacks can be mitigated, with explicit links to CWE and often implicitly to ATT&CK techniques mit [a].
- **MITRE ENGAGE.** A framework for adversary engagement strategies, which can be integrated into CTI graphs as a defensive complement to ATT&CK Cardei [2025].
- **MITRE D3FEND.** A complementary knowledge graph of defensive countermeasures structured to mirror the offensive ATT&CK ontology, providing a controlled vocabulary for the defensive techniques that map back to specific attack techniques.

Ariadne: The Athena CTI Knowledge Graph

Ariadne is the codename for the Athena CTI knowledge-graph substrate—the populated Neo4j database that all downstream Athena components (template generation, SFT, evaluation harness) consume as their single upstream source of CTI facts. The design premise is simple: every fact that can appear in an Athena SFT example must first exist as a node or edge in Ariadne. The graph is therefore both the training corpus's data layer and the evaluation harness's reference oracle, and the rigor of Athena's downstream guarantees rests on Ariadne's source coverage and refresh discipline.

Source coverage.

Ariadne ingests twelve public CTI sources spanning offensive technique catalogues, vulnerability and weakness databases, exploitation evidence, defensive countermeasures, and detection content:

- **MITRE ATT&CK** (Enterprise, STIX 2.1 JSON) and **MITRE ENGAGE** (JSON) for offensive and adversary-engagement technique catalogues.
- **MITRE CAPEC** (XML) and **MITRE CWE** (XML) for attack patterns and weakness classes.
- **MITRE D3FEND** (JSON-LD + SPARQL JSON, pinned to v1.4.0 for substrate stability) for defensive countermeasures aligned to ATT&CK.
- **CVE Project (CVE 5.0)** for the canonical vulnerability records.
- **NVD CPE/CVSS feeds** (per-year gzipped NDJSON) for the vendor-product affected-component and severity overlays.
- **CISA KEV** (Known Exploited Vulnerabilities catalogue, JSON) for the authoritative "actively exploited" signal.
- **FIRST EPSS** (gzipped CSV) for the daily exploitation-probability scores keyed to CVE identifiers.
- **Sigma rules** (YAML) for community detection content keyed to ATT&CK techniques.
- **ExploitDB** (CSV) and **PoC-in-GitHub** (JSON year folders) for the weaponized-exploit evidence layer associated with specific CVEs.

ATT&CK, ENGAGE, CAPEC, CWE, CISA KEV, D3FEND, and Sigma are loaded in their entirety because the catalogues themselves are bounded. CVE, NVD, EPSS, ExploitDB, and PoC-in-GitHub

are scoped to 2024 onwards. The cutoff is enforced both at clone time (sparse-clone path selection) and at parse time (per-source date filters), and is the principal mechanism by which Ariadne keeps the populated graph at a working size of approximately 280 000 nodes and 480 000 edges—large enough to support multi-hop, cross-schema reasoning over the modern threat surface, small enough to fit on a single Neo4j instance with practical query latency.

Refresh discipline.

The populator is idempotent: every Cypher write uses `MERGE` against per-node uniqueness constraints (`stix_id` for STIX-bearing nodes plus natural-key constraints on identifiers such as CVE IDs, CWE IDs, and D3FEND IDs), so re-running the populator never produces duplicates. The download layer applies three distinct refresh policies. *Always-refresh* sources (FIRST EPSS, NVD current-year feed) are re-downloaded on every populator run, because both update daily; EPSS specifically pulls the prior day's snapshot and overwrites the local CSV in place. *Skip-if-cached HTTPS* sources (CAPEC, CWE, CISA KEV, D3FEND) are downloaded once and re-used on subsequent runs unless the cached file is explicitly deleted; D3FEND additionally pins a version constant so substrate stability is byte-exact across runs until the version is bumped. *Skip-if-cached Git* sources (ATT&CK, ENGAGE, CVE, Sigma, ExploitDB, PoC-in-GitHub) are sparse-cloned once and refreshed by manual `git pull` or by deleting the source directory. The pattern that emerges in production is a nightly cron run that picks up the daily EPSS and current-year NVD churn while leaving the static catalogues untouched until their cache is manually cleared.

Graph construction order.

Ariadne is built in a fixed dependency order chosen to ensure that cross-framework edges always have both endpoints present when the relationship is created. Framework nodes are populated first—ATT&CK, CAPEC, CWE, ENGAGE, D3FEND, and Sigma—because these are the bounded catalogues that define the controlled vocabulary the rest of the graph references. The vulnerability surface follows: CVE records, NVD CPE/CVSS overlays, CISA KEV exploited-vulnerability flags, and FIRST EPSS probability scores. The weaponized-exploit layer (ExploitDB rows, PoC-in-GitHub entries) is last, since its rows reference CVE nodes that must already exist. Intra- and cross-framework relationships are then materialized, including the canonical CVE→CWE root-cause edges, CWE→CAPEC and CAPEC→ATT&CK pivots, ATT&CK→Mitigation and ATT&CK→Data-Component links, and the D3FEND↔ATT&CK defensive overlay.

Downstream consumption.

Ariadne is the upstream substrate for the template generation module described in Section 4.4, which traverses the graph to produce the IFT triples consumed by the SFT chain (Section 5.4). The same Neo4j connection parameters used by the populator are read by the template generator, ensuring that the database name (`athena-cti-db`) and authentication credentials are the single source of truth across the entire Athena pipeline. The fact that every SFT training row is materialized from a Cypher query against Ariadne is what makes the chained-SFT pipeline auditable in the sense discussed in Section 5.1: every answer in every training row is traceable to specific node and edge identifiers in the populated graph, which in turn are traceable to specific public-source rows by their natural keys.

Sophia: Graph-Template-Driven Instruction Generation

Pairing Ariadne is **Sophia** (`tmpl_gen`), the graph-template generator authored by Cardei and Khan that compiles declarative CTI reasoning patterns into Alpaca-format instruction triples Cardei [2025]. Where Ariadne is the data substrate, Sophia is the projection layer: it issues Cypher queries against the graph and binds the returned nodes, properties, and relationships into (instruction, input, output) triples that become Glaukopis training rows. Because every Sophia template resolves to typed graph traversals against named-key constraints in Ariadne, every triple is grounded in specific graph elements that can be cited by their natural keys (e.g., `CVE-2024-12345`, `T1059.003`, `CAPEC-66`). This is the structural feature on which the auditability claim of structured finetuning (Section 5.1) rests.

Sophia's key properties as a generator—taken collectively from the design document and its current implementation—are:

- **CTI source coverage.** Sophia binds against the same twelve sources ingested by Ariadne (Section 4.3), grouped for template-authoring purposes into three categories: (1) structural/graph sources (MITRE ATT&CK, CVE, CAPEC, CWE, MITRE ENGAGE, EPSS, CISA KEV, D3FEND), bound directly via Cypher; (2) LLM-assisted sources (Sigma rules, MITRE CAR, Wazuh rules), where natural-language fields are augmented by LLM extraction before being merged into the graph; and (3) proprietary sources (e.g., Flashpoint Ignite) where licensed feeds are normalized into the same node and edge schema.
- **Triple format and serialization.** Each generated data point is a three-field record with `instruction`, `input`, and `output` keys in standard Alpaca format: the instruction specifies the role and style (e.g., "You are a cyber security expert that gives precise responses to complex CTI questions"), the input is the user-facing CTI query, and the output is the ground-truth answer materialized from the CTI graph.
- **Template language.** Templates provide a high-level, declarative way to bind CTI graph nodes and produce triples. They support aliases for CTI types; attribute access through nested properties; filters with equality, inequality, and any/all semantics on list-valued properties; local variables bound to nodes; random selection from lists; list-comprehension-like constructs that expand relationships into output lists; invisible sections that establish bindings and constraints without appearing in surface text; and source disambiguation via a `source#type` prefix (e.g., `attck#attack-pattern` versus `capec#attack-pattern`) Cardei [2025].
- **Positive and negative examples.** The template language explicitly supports negative constraints (mitigations that do *not* mitigate a technique, data components that do *not* detect a technique used by a malware family), enabling systematic generation of hard negatives that teach models what does *not* follow from the CTI graph.
- **Cross-source reasoning templates.** Example templates compose multi-hop reasoning chains: given a CVE description, identify the corresponding CWE weakness, then find a CAPEC attack pattern with high likelihood of attack using that weakness, then find an ATT&CK technique that implements that attack pattern—each hop a typed query in Ariadne.

- **Incremental and time-windowed generation.** Sophia accepts version and date filters at both document and concept level: per-source version ranges and last-modified intervals can be specified in the job configuration, and relationships are only used if at least one endpoint and the relationship itself fall within those windows. This is what lets the training corpus track the threat landscape as new CTI material is ingested into Ariadne.

Operational details of the Sophia pipeline—the three-step `make_dataset.sh` workflow, the concrete template-syntax example, the active v21 template vintage at `templates/05182026/`, and the schema-validation tooling—are described in Section 5.1 where they sit alongside the SFT chain that consumes them.

Conceptually, the Ariadne + Sophia pairing is a symbolic counterpart to the LLM-assisted instruction pipelines used in CyberPal and SEvenLLM: where CyberPal synthesizes security instructions from unstructured sources using LLMs and human experts Levi et al. [2024, 2025], Athena uses a declarative template language over a typed CTI graph to generate strongly grounded and precisely controllable instruction triples.

CTI Knowledge Graphs and LLMs

Recent work shows that LLMs and CTI knowledge graphs are synergistic. Various efforts use LLMs to extract triples from CTI text and populate knowledge graphs, then perform link prediction and reasoning over those graphs to produce actionable recommendations Fieblinger et al. [2024], Zhang et al. [2023], Wu et al. [2022], Yu et al. [2024], Zhao et al. [2024]. Other systems build CTI-KGs directly from ATT&CK, CVE, threat reports, and open sources, then query them using LLMs or hybrid systems for tasks such as threat hunting and reporting.

The Ariadne + Sophia design follows a complementary pattern: *knowledge graph → templates → IFT triples → SLM*, with AthenaBench serving as a *knowledge graph → evaluation tasks* pipeline on the evaluation side. This alignment is central to the next section.

Glaukopis: Template-Based SFT Model for Athena-CTI-LLM

Glaukopis is the template-based supervised fine-tuning (SFT) pipeline used to produce the Athena-CTI-LLM family of cyber security expert small language models. It is the concrete instantiation of the design principles surveyed in earlier sections: a CTI knowledge graph drives a template generator (Section 4); the generator emits IFT triples that are sharded into purpose-built training datasets; those datasets feed a multi-stage chained SFT recipe (Section 2.6); and the resulting checkpoints are evaluated against a portfolio of cyber security benchmarks (Section 3). This section describes each of these elements and how they fit together.

Template-Based IFT: Sophia as the Training-Data Substrate

The Glaukopis training corpus is produced entirely by **Sophia** (`tmpl_gen`), the graph-template generator introduced in Section 4.4, operating against the Ariadne CTI substrate (Section 4.3). This subsection covers the operational side of Sophia—the pipeline shape, the template-language surface syntax, the Alpaca output format, the active v21 vintage, and the schema-validation tooling—and the scientific properties of the template-driven IFT corpus that motivate its use as the substrate for every Glaukopis SFT stage.

Pipeline shape

Sophia is a three-step pipeline driven by a single wrapper script, `data_generation/make_dataset.sh`, that runs end-to-end from authored templates to a Glaukopis-ready dataset:

1. `docx2json.sh`—extracts templates from the authoring format (Word `.docx` or JSON) into a structured intermediate JSON. Templates can be authored in either form: each `.docx` paragraph encodes one template with inline `Id Instruction: ...Question: ...Answer: ...` markers, or each JSON object carries explicit `id`, `instruction`, `question`, `answer` fields.
2. `tmpl2triples.sh`—reads the template JSON, opens a Bolt connection to Ariadne using the credentials in `neo4j-local-config.json`, issues the implied Cypher queries for each template, and emits per-template triple files in the configured results directory. A `_results-report.json` summary captures `failed_count` together with per-template generation counts so authors can diagnose binding failures.
3. `triples2alpaca.sh`—merges all per-template triple files into a single Alpaca-format JSON dataset suitable as direct input to the SFT chain.

The configurable knobs of the wrapper are a per-template generation ceiling (`count_limit`, default 10) controlling how many distinct bindings are sampled per template at the docx-to-JSON stage, and a maximum triple count (`count_max`, default 2,000) controlling the upper bound on triples emitted per template at the generation stage. These knobs are the per-stage shard-sizing controls used by the v21 build recipe in Section 5.3.

Template syntax

A Sophia template binds graph nodes to local aliases, references their typed properties through dotted attribute access, and produces natural-language output containing the resolved values. A representative template (excerpted from the v21 manifest) reads:

```
M.1 Instruction: You are a cyber security expert that has been trained
to give precise responses to complex CTI questions. You work in a
SOC protecting data for enterprise customers. Question: Explain ATT&CK
technique {ap:attack-pattern.name} ({ap.id}) and how it is used.
Answer: Technique {ap.name} ({ap.id}) is described as follows: {ap.descript
It is commonly observed across {ap.x_mitre_platforms}.
```

Here `{ap:attack-pattern.name}` introduces `ap` as a local alias for an ATT&CK attack-pattern node and emits its `name` property; subsequent references `{ap.id}`, `{ap.description}`, and `{ap.x_mitre_platforms}` reuse the same binding and emit additional properties of the same node. The full language adds relationship traversal (`{var1.rel>TargetType.property}`), filters and any/all semantics on list-valued properties, list-comprehension constructs that expand relationships into output lists, invisible sections that establish bindings without producing surface text, and source-disambiguating prefixes (`attck#attack-pattern` vs. `capec#attack-pattern`). The detailed grammar is documented in Cardei's IFT design document Cardei [2025].

Output format

Each row of the Sophia output file is a three-key JSON object in standard Alpaca format, with `instruction` carrying the system role, `input` carrying the question derived from the template, and `output` carrying the answer materialized from the graph:

```
{
  "instruction": "You are a cyber security expert...",
  "input":       "Explain ATT&CK technique Phishing (T1566) and how
it is used.",
  "output":      "Technique Phishing (T1566) is described as follows:
Adversaries may
       send phishing messages to gain access to victim systems. It
is commonly
       observed across Windows, macOS, Linux."
}
```

The Alpaca shape is the canonical SFT input format used by LlamaFactory, AutoTrain, and the related open-source SFT tooling that the Glaukopis training scripts wrap; no further reformatting is required between Sophia output and SFT consumption.

Active vintage and provenance

The active template vintage at the time of writing is **v21**, residing under `tmpl_gen/templates/05182026/` as three manifests—a Core manifest, a TAA manifest, and a CSE manifest—feeding the four-stage chained SFT recipe of Section 5.4. The v21 vintage is a strict-reproducibility experiment forked from the v18.1 vintage; the templates and per-stage row-count gates are byte-identical to v18.1, with only dataset names and Hugging Face push targets changed (Section 5.3).

Schema-validation tooling

A separate `schema-test/` module accompanies Sophia and operates against the same Ariadne instance: it auto-generates test templates from a target CTI schema XLSX (`make-test-templates.sh`), runs them through the triple-generation engine (`test-CTI-schema.sh`), and emits a results report that flags any node label, property, or relationship that is referenced by a template but absent from the deployed graph. This is the mechanism by which Ariadne schema drift (e.g., a renamed property in a refreshed source) is detected before it can silently break template-generated training rows.

Scientific properties of template-driven IFT

Sophia's design properties translate into several scientific advantages that motivate its use as the substrate for every Glaukopis SFT stage relative to purely LLM-generated instruction datasets:

- **Ground-truth faithfulness.** Answers are computed directly from an authoritative CTI graph, reducing hallucinations and inconsistencies in training labels relative to instruction sets generated by an LLM acting as both author and oracle.
- **Explicit coverage control.** Templates can be designed to cover specific combinations of entities (e.g., CAPEC patterns affecting memory resources and their detection methods) or relationships (e.g., malware using two techniques on Linux), and per-template coverage fractions and count limits control dataset size and diversity Cardei [2025].
- **Rich negative examples.** Inequality constraints and relationship filters enable systematic generation of hard negatives such as mitigations that do *not* address a technique or data components that do *not* detect any technique used by a malware family, exercising the model's ability to discriminate rather than simply complete.
- **Auditability and explainability.** Every Alpaca row maps to a specific template plus a specific graph binding; security experts can audit individual training rows by re-running the implied Cypher query, and changes to a template propagate deterministically to all rows it produced.
- **Incremental refresh.** Version and date filters allow periodic regeneration of updated IFT corpora as CTI sources evolve, paralleling AthenaBench's dynamic evaluation but on the training side Cardei [2025], Alam et al. [2025].

These properties together support a refreshable training loop in which new CTI material in Ariadne, new templates in Sophia, regenerated dataset shards, re-run SFT stages, and benchmark-driven feedback are all separable operations rather than a single monolithic retrain. The v21 vintage represents one concrete instantiation of this loop; the next vintage will refresh against an updated Ariadne snapshot and is expected to incorporate seed-pinning fixes to the substrate sampler discussed in Section 5.3.

Status.

Sophia is in active development. Template syntax, parser behavior, and schema-test tooling may evolve as the CTI graph and the v21+ training pipeline mature; the descriptions in this section reflect the implementation as of the v21 vintage.

Comparison with IBM's CyberPal and Other Instruction Pipelines

The Athena approach is conceptually close to SecKnowledge/CyberPal, but with a different emphasis:

- **CyberPal / SecKnowledge.** Uses expert-defined schemas and a pipeline combining LLMs and scripts to synthesize instructions from CTI sources and detection artifacts (e.g., Sigma rules), emphasizing human-curated "expert instruction patterns" and LLM-assisted generation, including CoT answers in SecKnowledge 2.0 Levi et al. [2024, 2025].
- **Sophia over Ariadne.** Uses a symbolic template language compiled into Cypher queries against the Ariadne CTI graph, producing triples that are guaranteed consistent with the knowledge graph, with strong use of cross-schema constraints and negative examples, and built around incremental, configuration-driven generation Cardei [2025].

The two approaches are complementary. CyberPal demonstrates the effectiveness of expert instruction schemas and CoT enrichment in achieving high cyber security performance, especially in SLMs. The Ariadne + Sophia pairing provides precise and reproducible control over content generation aligned with CTI schemas and knowledge-graph structure, potentially feeding into similar CoT-enriched pipelines (e.g., by later adding CoT rationales using LLMs on top of graph-derived answers).

The v21 Template-Generation Vintage

The Glaukopis SFT chain consumes training data produced by a versioned template-generation vintage. The active vintage at the time

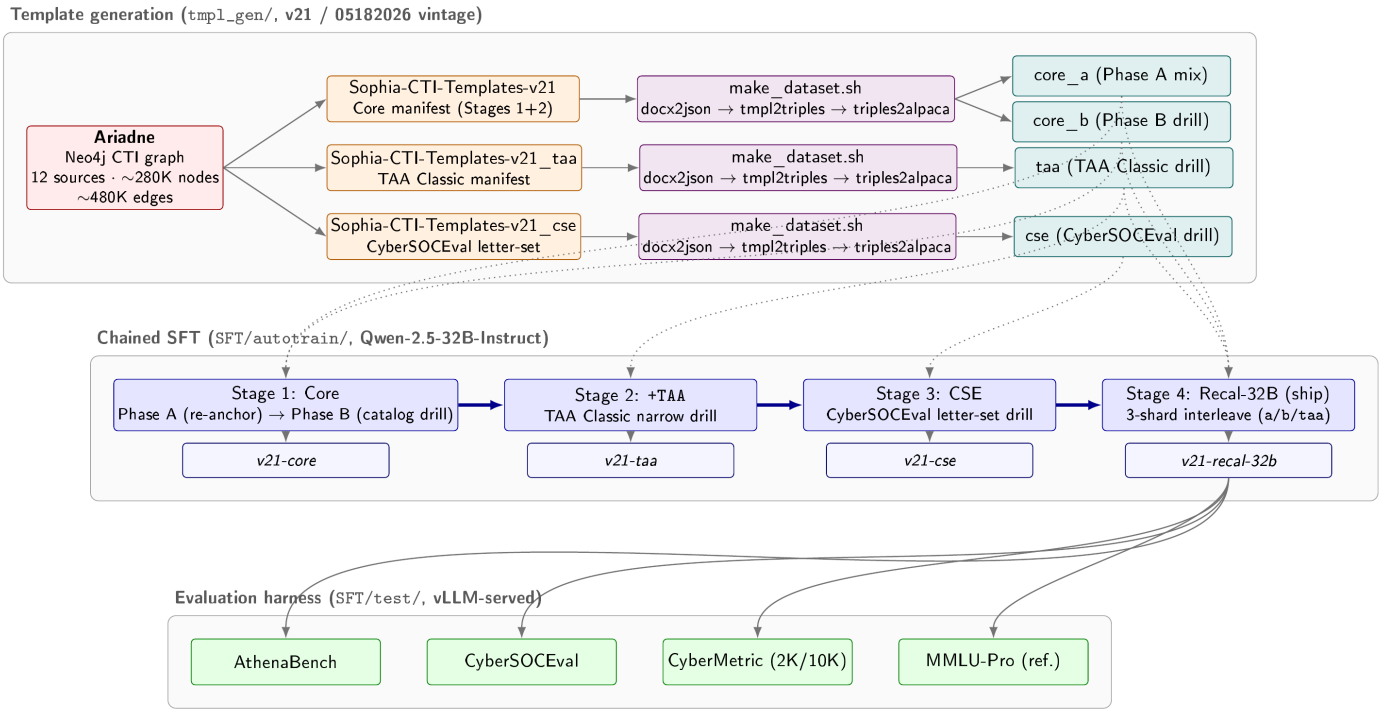

Figure 1 Glaukopis v21 end-to-end flow. Ariadne (Section 4.3), the Athena Neo4j CTI substrate, feeds three v21 template manifests—Core, TAA, and CSE—each processed by the `make_dataset.sh` pipeline into Alpaca-format dataset shards. Four production shards (`core_a`, `core_b`, `taa`, `cse`; matched validation shards omitted for clarity) drive the four-stage chained SFT recipe targeting Qwen-2.5-32B-Instruct, producing the cumulative `v21-core`, `v21-taa`, `v21-cse`, and ship `v21-recal-32b` checkpoints. Dotted arrows mark which shards feed which stages; Stage 4 re-uses Core-A, Core-B, and TAA as a three-shard interleaved touch-up. Each pushed checkpoint is evaluated against the AthenaBench, CyberSOCEval, CyberMetric, and MMLU-Pro suites under a vLLM-served harness.

of writing is **v21** (template directory `tmpl_gen/templates/05182026/`), which is the configuration used for all hyperparameters, dataset structure, and results discussed in the rest of this section. This subsection describes the v21 template-generation flow—how Ariadne (Section 4.3) is projected through three template manifests into the seven dataset shards consumed by the four-stage SFT chain (Section 5.4)—and the reproducibility and lineage discipline that produced it.

Figure 1 summarizes the end-to-end pipeline. Ariadne feeds three template manifests: a *Core* manifest covering Stages 1 and 4, a *TAA* manifest covering Stage 2, and a *CSE* manifest covering Stage 3. Each manifest is processed by the same three-step `make_dataset.sh` pipeline—`docx2json` extracts templates from their authoring format, `tmpl2triples` drives the IFT-generation engine against the Neo4j graph, and `triples2alpaca` merges per-template triple JSON files into a single Alpaca-format dataset (with `instruction`, `input`, and `output` keys mapping to system, prompt, and response respectively). The result is four production training shards plus three matched validation shards, all template-baked and therefore architecture-independent: a single set of seven JSON files feeds every base-model port of the v21 recipe.

Reproducibility provenance.

v21 is a strict reproducibility experiment on top of a predecessor vintage v18.1. The three Sophia-CTI template manifests, the row-count gate JSONs, and the per-stage count limits are `shasum`-verified byte-identical to their v18.1 sources; only dataset names, build-directory names, and Hugging Face push targets change between the two vintages. The v21 sign-off gate was that the post-Core checkpoint would land within ±1.5 percentage points of the v18.1 Core peak on every benchmark axis. The Total Score landed inside that band, but two per-axis sub-scores (ATE and RCM) drifted outside it, with the same shape that the intermediate v19 and v20 vintages had previously shown against v18.1. The most parsimonious explanation, given that the recipe-side inputs are byte-identical, is data-build non-determinism in the substrate sampler, dedup tiebreaks, and shuffle seeding inside `make_dataset.sh`—a hypothesis confirmed by the fact that pinning the Neo4j read order and seeding the substrate sampler is the next-vintage research direction. The lineage discipline is in our judgement itself a contribution: by carrying byte-identical templates and gates forward across vintages, the Glaukopis pipeline isolates recipe drift from data-layer drift as separable failure modes, and provides a falsification matrix against which future vintages can be evaluated.

The off-plan Recalibrate stage.

The v21 plan as originally specified was a three-stage chain (Core → +TAA → CSE). A recurring pattern observed across v18.1, v19, v20, and v21 chains is that Stage 3's narrow CyberSOCEval-letter-set drill reliably trades approximately ten percentage points of VSP for the CyberSOCEval-Malware lift it is designed to produce. To recover the VSP regression without sacrificing the Stage 3 gains, an off-plan fourth stage—*Recalibrate*—was added as a three-shard

interleaved touch-up over Core-A, Core-B, and TAA. At the 14B base scale, a single Recalibrate recipe (1e-6 LR, 0.25/0.40/0.35 interleave probabilities, `--max-samples 2400`) cleanly recovers VSP. At the 32B base scale, the same recipe sits at the `adamw_8bit` optimizer noise floor and drifts VSP the wrong way; the recipe was therefore re-tuned (3e-6 LR, 0.15/0.60/0.25 Phase-B-heavy mix, `--max-samples 3600`) to expose more VSP/RMS catalog per interleaved row while holding step count and wall-time constant. The 32B Stage 4 split is the basis of the ship checkpoint discussed in Section 5.4.

Architecture ports.

Stages 1 through 3 carry the same recipe byte-identically across all five base-model ports of v21: Qwen-2.5-32B-Instruct (canonical), Qwen-2.5-14B-Instruct, Foundation-Sec-8B, Llama-3.1-8B-Instruct, and Qwen3-30B-A3B-Thinking-2507 (a thinking-mode MoE). The fact that the same template-baked shards drive all ports is what makes cross-architecture comparison meaningful: any axis-level differences between checkpoints derived from different bases under the same recipe and same data are attributable to base-model differences rather than training-data drift. The Qwen3-MoE port is also the first port to exercise a thinking-on / thinking-off A/B at serve time, exposing the SFT contribution to no-trace inference behavior separately from the base model's reasoning gain. Stage 4 is the only stage where the recipe does not carry identically across architectures: at the 32B dense scale the Phase-B-heavy 3e-6 LR variant is required (as discussed above), and on the Qwen3-MoE port Stage 4 closed without a beat against `v21-cse` on either Stage-4 variant, so the on-chain ship for that port is `v21-cse`.

The Glaukopis SFT Chain

The Glaukopis SFT pipeline is a four-stage chain over template-generated CTI instruction data. The canonical configuration targets Qwen-2.5-32B-Instruct as the base model and produces a sequence of four cumulative checkpoints—Core, +TAA, CSE, and Recalibrated-32B—of which the last is the production ship checkpoint. The same recipe ports byte-identically to four other base architectures (Qwen-2.5-14B-Instruct, Foundation-Sec-8B, Llama-3.1-8B-Instruct, and the Qwen3-30B-A3B-Thinking-2507 MoE), with only the base-model pointer and per-architecture memory flags changing between ports. The four-stage structure is itself the load-bearing element of the recipe; the per-stage details are described below.

Dataset shards.

The chain consumes seven dataset shards produced by the template generator. The **Core-A** shard is a broad re-anchor mix combining knowledge-base, multiple-choice, threat-actor-attribution (TAA), SOC-style, CyberMetric-style, mitigation-strategy, and yes/no instruction families. The **Core-B** shard is an AthenaBench catalog drill targeting Risk Mitigation Strategy (RMS), Attack Technique Extraction (ATE), Vulnerability Severity Prediction (VSP), and Root Cause Mapping (RCM) tasks Alam et al. [2025]. The **TAA** shard is a narrow drill on TAA Classic. The **CSE** shard is a CyberSOCEval letter-set drill, structured to exercise the format and reasoning pattern used by the CyberSOCEval malware and threat-intel test suites Meta AI / Purple Llama Team [2025]. Three matched validation shards (**Core-Val**, **TAA-Val**, **CSE-Val**) hold out a per-axis slice (50 rows per shortname) for held-out evaluation during training. All shards are template-baked and therefore architecture-independent: a single set of seven JSON files feeds every base-model port.

Stage 1: Core (broad re-anchor + AthenaBench catalog drill).

Stage 1 runs as two phases inside a single launcher, with only the final phase's merged checkpoint pushed. *Phase A* is the broad re-anchor: it trains on the Core-A shard together with a general-purpose instruction mix (the Tülu-3 SFT mixture and an Alpaca-English demonstration set) at sequence length 8 192, packing on, learning rate 1×10^{-5}, and effective batch size 16. The intent is to re-establish broad instruction-following behavior on top of a CTI vocabulary base. *Phase B* drills on the Core-B AthenaBench-catalog shard at sequence length 16 384 with packing off (so that long structured outputs are not truncated by packing boundaries), learning rate 5×10^{-6}, and effective batch size 8. The shorter, lower-LR Phase B is calibrated to imprint precise AthenaBench task structure on top of the broad foundation laid by Phase A, rather than to continue broad assimilation Alam et al. [2025]. Together, Phase A and Phase B implement at fine granularity the principle from FLAN-T5 and "Don't Stop Pretraining" that broad domain exposure and narrow task supervision benefit from separate optimization regimes Chung et al. [2022], Gururangan et al. [2020].

Stage 2: +TAA narrow drill.

Stage 2 takes the Core checkpoint as its base and trains on the TAA shard alone, at sequence length 4 096, packing on, learning rate 5×10^{-6}, and effective batch size 16. The objective is to harden the model's TAA Classic performance without disturbing the broader catalog skills established in Stage 1.

Stage 3: CSE letter-set drill.

Stage 3 takes the +TAA checkpoint and trains on the CSE shard at the same sequence length, packing, learning rate, and effective batch size as Stage 2. This stage is targeted preparation for the CyberSOCEval malware and threat-intelligence suites Meta AI / Purple Llama Team [2025], and is the point at which a known trade-off appears: lifting the CyberSOCEval axis through this drill comes at a measured cost to VSP performance from Stage 1's Phase B, which the next stage exists to recover.

Stage 4: Recalibrated touch-up (ship variant).

Stage 4 is a three-shard interleaved touch-up trained off the Stage 3 (CSE) checkpoint with the explicit goal of recovering the VSP regression introduced by Stage 3 without losing Stage 3's CyberSOCEval gains. The ship variant trains on a Core-A / Core-B / TAA mix at relative sampling probabilities 0.15/0.60/0.25 (i.e., Phase-B-heavy), using LlamaFactory's `interleave_under` mixing strategy with a 3 600-row sample cap, at sequence length 16 384, packing off, learning rate 3×10^{-6}, and effective batch size 8. The 14B-tuned variant of this recipe (lower learning rate, more balanced mixture, smaller sample cap) is retained as a recipe-provenance A/B against the ship-grade 32B recipe, but in the 32B-target chain it is the higher-LR, Phase-B-heavy variant that recovers VSP and tops the v21 evaluation leaderboard.

Cross-cutting training-system details.

All four stages share the same systems-level configuration: bf16 mixed precision, DeepSpeed ZeRO-3 (with optional CPU offload), the

`adamw_8bit` optimizer (mandatory at the 32B scale to fit activations), Liger kernels for fused attention paths, and gradient checkpointing on at 32B regardless of GPU count. The 8×H100-80GB SXM topology is the canonical training target; 8×H200-141GB SXM and 8×RTX-PRO-6000-96GB are supported alternates. Effective batch size is held constant across launch topologies by auto-scaling gradient accumulation to GPU count.

Why four stages rather than one.

The four-stage structure embodies the chained-SFT rationale from Section 2.6 in a CTI-specific form. Stages 1A and 1B separate broad domain absorption from precise structured-task imprinting, each at its own learning rate and packing regime Gururangan et al. [2020], Chung et al. [2022]. Stages 2 and 3 carry out narrow drills against specific benchmark axes (TAA and CyberSOCEval respectively) without re-perturbing the broader skill set learned in Stage 1. Stage 4 is a recalibration step that recovers regressed axes (here, VSP) through controlled three-shard interleaving and a slightly higher learning rate, exploiting the fact that all template-generated shards remain available as training corpora throughout the chain. The result is a sequence of intermediate checkpoints—Core, +TAA, CSE, Recalibrated-32B—that can each be evaluated independently against the benchmark portfolio of Section 3, supporting controlled per-stage ablation in addition to producing a single ship checkpoint. This per-stage observability is itself a benefit of the chained structure: any axis-level regression observed at evaluation time is attributable to a specific stage and a specific dataset shard, and can be addressed by re-running only the affected stages rather than the full pipeline.

Evaluation Harness

Each pushed checkpoint in the Glaukopis chain is evaluated against the portfolio described in Section 3, consisting of **AthenaBench** (the six CTI tasks: CKT/MCQ, RCM, VSP, ATE, TAA Classic and Canonical, and RMS) Alam et al. [2025]; **CyberSOCEval** (the malware and threat-intelligence test suites) Meta AI / Purple Llama Team [2025]; **CyberMetric** at the 2 000- and 10 000-question tiers Tihanyi et al. [2024]; and **MMLU-Pro** as a decoupled general-intelligence reference. Serving is via vLLM in an OpenAI-compatible configuration; the same alias-keyed harness scores public baselines through hosted inference providers for direct comparison. Per-task token and wall-clock usage is captured so that benchmark sweeps are accompanied by a cost summary covering both API and local-GPU compute, which lets evaluation cost be reported alongside accuracy as a first-class metric.

The mapping between training stages and evaluation suites is deliberate: AthenaBench RMS/ATE/VSP/RCM are exercised by Stage 1 Phase B's Core-B shard; AthenaBench TAA Classic and TAA Canonical are exercised by Stage 2; CyberSOCEval malware and threat-intel reasoning are exercised by Stage 3; CyberMetric and the AthenaBench MCQ axis are exercised primarily by Stage 1 Phase A's Core-A shard. The Stage 4 recalibration is designed not to alter this mapping but to recover any axis that has regressed across the prior stages, with the 32B-tuned, Phase-B-heavy mixture specifically chosen to restore VSP without sacrificing CyberSOCEval performance. MMLU-Pro is run on a separate session so that suite scope stays decoupled from CTI scoring: it is reported as a sanity check on general capability rather than as a target for optimization.

Toward Cyber Security Expert Small Language Models

Bringing together the elements above, an end-to-end cyber security expert SLM pipeline of the kind exemplified by Glaukopis can be summarized as:

1. **CTI graph construction.** Continuously ingest and normalize CTI sources (ATT&CK, CVE, CAPEC, CWE, ENGAGE, ICS/OT sources, proprietary feeds) into a Neo4j or similar graph.
2. **Template authoring.** CTI experts encode important reasoning patterns (knowledge recall, root cause mapping, severity estimation, attribution clues, mitigation reasoning) as templates in the Athena language.
3. **IFT generation.** Run the generation tool with version and date constraints aligned to a training window, producing large, structured IFT datasets per task family Cardei [2025].
4. **Chained SFT.** Fine-tune SLMs (e.g., 8–32B open models) using a chain of stages as described in Section 5.4, with the dataset-shard / stage mapping aligned to the benchmark portfolio.
5. **Evaluation.** Benchmark models on AthenaBench, CyberSOCEval, CyberMetric, and complementary suites (CTIBench, SecEval, CyberBench, SEvenLLM-Bench) to assess generalization and compare with larger proprietary models Alam et al. [2024, 2025], Hu et al. [2024], Liu et al. [2024], Ji et al. [2024], Tihanyi et al. [2024], Meta AI / Purple Llama Team [2025].
6. **Feedback and iteration.** Use benchmark error patterns to design new templates, adjust CTI graph coverage, or refine IFT data (e.g., adding more negative cases or multi-hop chains), and re-run only the affected SFT stages.

This pipeline is closely aligned with IBM's vision of cyber security expert SLMs Levi et al. [2024, 2025], but anchored in Athena's graph-driven IFT, the chained-SFT recipe described above, and dynamic CTI benchmarking via AthenaBench and CyberSOCEval.

Results

This section reports the evaluation of the v21 Glaukopis vintage against the benchmark portfolio of Section 3: AthenaBench (six tasks: CKT, ATE, RCM, RMS, VSP, and TAA, with TAA scored as the combined-form average across Classic and Canonical), CyberSOCEval (Malware-analysis and Threat-Intel-Reasoning subsets), CyberMetric at the 2 000- and 10 000-question tiers, and MMLU-Pro as a decoupled general-intelligence reference. All numbers in this section come from the same harness run (sweep dated 2026-05-24), use the same vLLM serving stack for local checkpoints, and use the same hosted-API endpoints for public-model baselines.

Three Averaging Schemes and Why MMLU-Pro Is Reported Separately

Because the three CTI benchmarks differ substantially in their internal structure—AthenaBench has six tasks, CyberSOCEval has two, and CyberMetric has two tiers—there is no single canonical way to aggregate them into a summary number, and the choice of aggregation visibly changes the leaderboard. We therefore report three averages for every model:

- **Avg CTI Benchmark Score.** The unweighted mean of three per-benchmark summaries: AthenaBench (combined),

CyberSOCEval (combined), and CyberMetric (2K+10K average). Each benchmark contributes equal weight regardless of how many internal tests it contains. This is the closest analogue to how published leaderboards typically present cyber LLM results and is the score we use for cross-benchmark comparison.

- **Avg CTI Test Score.** The unweighted mean across the ten individual CTI tests—six AthenaBench tasks, two CyberSOCEval subsets, and the two CyberMetric tiers—giving each test equal weight. This view is sensitive to the internal structure of each benchmark: AthenaBench, with six tests, exerts more pull on the score than CyberSOCEval or CyberMetric, which have two tests each. It is the most informative when the goal is to compare aggregate per-task capability.
- **Avg Total Test Score (with MMLU-Pro).** The same per-test average as above, but extended to include MMLU-Pro. MMLU-Pro is a general-intelligence reference and is *not* part of the CTI use case we ultimately care about; we report this third average chiefly to quantify how much general capability each SFT run has traded away for CTI specialization—a *general-knowledge erosion* diagnostic (often called *catastrophic forgetting* in the SFT literature) rather than a target for optimization.

The three averages tell complementary stories. The per-benchmark average is the most charitable view for SFT models, because it rewards strong performance on benchmarks whose internal structure aligns with the training data (here, AthenaBench). The per-test average reweights toward AthenaBench's six tasks but is invariant to how those tasks are pooled, making it the most apples-to-apples cross-architecture metric. The MMLU-Pro-augmented average is informative only as a counterpoint: SFT'd models systematically lose MMLU-Pro relative to their bases, and the magnitude of that loss is a useful diagnostic for the cost of specialization.

Note on cost figures.

A per-evaluation cost analysis is in preparation alongside this manuscript. Costs are tracked end-to-end by the evaluation harness for both hosted-API models (token-priced via the published rate card of each provider) and local checkpoints (wall-clock × GPU rate on the 2×H100 reference host). The cost numbers reported in Section 6.5 are draft and subject to refinement before final submission; the headline accuracy and capability findings are stable.

Headline Comparison: v21 Ship Checkpoint vs. Frontier Reference Points

Table 1 compares the Glaukopis v21 ship checkpoint, `athena-cti-sft-qwen25-32b-v21-recal-32b`, against its base instruct model and against six frontier-class reference points spanning the major proprietary and open-weights frontier families. All three averages are reported alongside the underlying per-benchmark scores. Three observations:

1. **Chained SFT adds 7.3 points of average CTI benchmark capability over the base Qwen-2.5-32B-Instruct** (59.0 → 66.3) without changing the parameter count or runtime footprint. On the per-test average the lift is even larger (49.6 → 62.9, +13.3 points), because the per-test average gives full weight to AthenaBench's six tasks, on which the chain is explicitly trained. The largest single-axis lift is on AthenaBench combined (44.2 → 66.5; +22.3 absolute).
2. **The ship checkpoint approaches frontier-class CTI performance.** On Avg CTI Benchmark Score the v21 ship sits within 4–7 points of the leading frontier models (GPT-5.5 at 72.9, DeepSeek-V4-Pro at 70.4, Gemini-3-Flash-Preview at 69.8), and is closely competitive with mid-tier frontier offerings (GPT-5.2 at 67.5, DeepSeek-v3.2-Exp at 66.0). On Avg CTI Test Score the same pattern holds (62.9 vs. leaders at 67–73). It is competitive with Gemini-2.5-Flash on both averages despite the substantial parameter-count gap.
3. **General-knowledge erosion in the MMLU-Pro dimension is real, but not unique to Glaukopis and not a target for optimization.** MMLU-Pro drops from 69.0 (Qwen-2.5-32B-Instruct base) to 57.4 (v21 ship), an 11.6-point absolute regression that reflects the well-known cost of domain specialization. The Avg Total Test Score (with MMLU-Pro) figure makes this explicit: the v21 ship at 62.8 trails GPT-5.5 (76.9) by a wider margin on this composite than on either CTI-only average. The phenomenon is conventionally labeled *catastrophic forgetting* in the SFT literature; we prefer the more accurate term *general-knowledge erosion*, because what occurs in practice across our ports is a graded, parameter-count-dependent degradation of out-of-domain capability rather than a sudden, total loss. The graded pattern is visible in Table 3: the 8B ports lose 17.6–20.6 MMLU-Pro points, the 14B port loses 13.9, the dense 32B port loses 11.6, and the Qwen3-MoE port (largest active-parameter count among ports we ran) loses the most at 24.6—an outlier driven by the MoE architecture rather than by the base-model size. The dominant trend among dense ports is that larger base models erode less under the same recipe. Because MMLU-Pro is not part of the CTI use case we ultimately care about, this erosion is reported as a diagnostic rather than treated as a metric to optimize against; we discuss possible mitigations, including the option of moving to a larger base model, in Section 7.

Table 1 Headline results: Glaukopis v21 ship checkpoint vs. base model and frontier reference points. Three averages are reported per the schemes defined in Section 6.1: **Avg CTI Bench** (3 benchmarks, equal weight), **Avg CTI Test** (10 CTI tests, equal weight), and **Avg Total (w/MMLU)** (10 CTI tests plus MMLU-Pro). All numbers from the 2026-05-24 evaluation sweep. AB Avg II is the AthenaBench composite using the 50/50 TAA Classic+Canonical blend.

Model	Type	Avg CTI Bench	Avg CTI Test	Avg Total (w/MMLU)	AB Avg II	CSE comb	CM avg	MMLU-Pro
GPT-5.5	Frontier	72.9	73.5	76.9	77.0	48.3	93.3	88.3
DeepSeek-V4-Pro	OS-PT	70.4	67.9	69.6	67.2	52.2	91.8	79.1
Gemini-3-Flash-Preview	Frontier	69.8	68.4	71.7	69.3	48.1	92.1	89.2
GPT-5.2 (reasoning=medium)	Frontier	67.5	65.0	67.0	63.8	46.8	91.8	81.3
DeepSeek-v3.2-Exp	OS-PT	66.0	62.2	63.9	59.0	47.2	91.7	82.5
v21-recal-32b (ship)	SFT	**66.3**	**62.9**	**62.8**	**66.5**	42.6	89.8	57.4
v21-cse	SFT	65.8	62.4	62.2	66.6	42.0	88.8	54.2
v21-recalibrate (14B-recipe)	SFT	65.9	62.4	62.2	65.9	42.4	89.2	54.6
Gemini-2.5-Flash	Frontier	63.4	57.9	59.8	53.1	46.2	91.0	82.1
v21-core	SFT	62.6	61.6	63.2	67.2	31.1	89.6	59.2
v21-taa	SFT	62.0	61.7	63.7	66.9	29.8	89.4	58.2
Qwen2.5-32B-Instruct (base)	Instruct	59.0	49.6	49.3	44.2	42.6	90.2	69.0

Why `v21-recal-32b` is the ship checkpoint, not `v21-recalibrate`.

The two Stage-4 variants warrant brief comparison because they share the same Stage-3 parent (`v21-cse`) and differ only in the Stage-4 recipe. `v21-recalibrate` runs the 14B-tuned Stage-4 recipe verbatim on the 32B base (learning rate 1×10^{-6}, balanced 0.25/0.40/0.35 interleave probabilities, `--max-samples 2400`); `v21-recal-32b` runs the recipe re-tuned for the 32B parameter scale (learning rate 3×10^{-6}, Phase-B-heavy 0.15/0.60/0.25 mix, `--max-samples 3600`). On the headline aggregates the two variants are within 0.5 points of each other (Avg CTI Bench 66.3 vs. 65.9; Avg CTI Test 62.9 vs. 62.4), which on its own would not be enough to prefer one over the other. The deciding factor is per-axis: `v21-recal-32b` restores VSP to 77.8 against `v21-recalibrate`'s 75.7, and that VSP recovery is precisely what Stage 4 is for at the 32B scale (Section 5.3). The 14B-recipe variant sits at the `adamw_8bit` optimizer noise floor at 32B—enough to nudge most axes in the right direction, but not enough to recover the VSP regression introduced by Stage 3. `v21-recal-32b` is therefore the recommended ship checkpoint at the 32B scale, with `v21-recalibrate` retained on disk as the recipe-provenance A/B against it.

Per-Stage Ablation: What Each SFT Stage Contributes

Because the Glaukopis chain pushes a Hugging Face checkpoint after every stage, the per-stage contribution to each benchmark axis is directly measurable. Table 2 reports the AthenaBench axis breakdown for the Qwen-2.5-32B chain alongside the base model, decomposing how Core → +TAA → CSE → Recalibrate-32B accumulates capability.

The data confirm the design rationale of the chain (Section 5.4):

- **Stage 1 (Core) does most of the AthenaBench-catalog work.** The Phase B catalog drill lifts ATE from 18.2 (base) to 67.2 (Core), RMS combined-f1 from 7.6 to 63.8, and RCM from 59.5 to 69.8. These are the gains the rest of the chain must preserve.
- **Stage 2 (+TAA) introduces meaningful Canonical-TAA capability.** TAA Canonical combined rises from 2.0 (base) and 18.4 (Core) to 36.8 at +TAA, the largest single-stage Canonical lift in the chain.
- **Stage 3 (CSE) lifts the CyberSOCEval axis** from 31.1 (Core) to 42.0 (CSE), a +10.9 absolute gain. The expected VSP trade-off is also visible: VSP drops from 83.1 (Core) to 78.9 (CSE), consistent with the regression pattern observed across v18.1, v19, v20, and v21 (Section 5.3).
- **Stage 4 (Recalibrate-32B) is recovery, not novel capability.** On the headline Avg CTI score the ship checkpoint at 66.3 is essentially tied with the post-CSE checkpoint at 65.8 and the 14B-recipe recalibrate variant at 65.9; the Stage 4 lift comes from re-aligning VSP (77.8) and stabilizing the CyberSOCEval combined score (42.6) rather than from new capability development.

Table 2 Per-stage AthenaBench breakdown for the Qwen-2.5-32B v21 chain (combined-form scores where multiple forms are released). Stage 1 (Core) does most of the AthenaBench-catalog work; Stage 2 introduces Canonical TAA; Stage 3 lifts CyberSOCEval; Stage 4 recovers VSP without losing prior gains.

Checkpoint	CKT	ATE	RCM	RMS	VSP	TAA Cl.	TAA Can.	AB II
Qwen2.5-32B-Instruct (base)	52.9	18.2	59.5	7.6	78.8	48.0	2.0	44.2
v21-core	72.9	67.2	69.8	63.8	83.1	46.5	18.4	67.2
v21-taa	75.4	63.4	69.0	62.9	83.5	47.0	36.8	66.9
v21-cse	72.5	63.4	67.7	63.7	78.9	53.5	3.4	66.6
v21-recal-32b	75.5	65.8	67.1	62.5	77.8	50.5	3.4	**66.5**

Table 3 Cross-architecture v21 ports. *Best v21*: highest-scoring SFT checkpoint for that base architecture on Avg CTI Benchmark Score. **Avg CTI Bench** and **Avg CTI Test** are the per-benchmark and per-test schemes defined in Section 6.1. Δ *test* and Δ *MMLU*: absolute differences between the best SFT checkpoint and the corresponding instruct base on Avg CTI Test and MMLU-Pro respectively. The Qwen-2.5-32B dense port is the cross-architecture optimum on CTI; the Qwen3-MoE port leads on MMLU-Pro among SFT'd checkpoints but is more expensive to serve and benchmark.

Base architecture	Best v21 checkpoint	Avg CTI Bench	Avg CTI Test	Δ test vs. base	MMLU-Pro	Δ MMLU vs. base
Qwen2.5-32B-Instruct	v21-recal-32b	**66.3**	**62.9**	+13.3	57.4	−11.6
Qwen2.5-14B-Instruct	v21-recalibrate	62.3	59.6	+13.5	50.2	−13.9
Qwen3-30B-A3B-Thinking-2507	v21-cse	63.4	60.9	+6.0	53.0	−24.6
Foundation-Sec-8B-Instruct	v21-recalibrate	53.8	53.2	+0.4	25.6	−17.6
Llama-3.1-8B-Instruct	v21-recalibrate	50.5	50.8	+7.3	27.7	−20.6

Cross-Architecture Ports: Recipe Transfer

The same v21 recipe ports byte-identically to four other base architectures, with only the base-model pointer and per-architecture memory flags changing between ports (Section 5.3). Table 3 compares the best v21 SFT checkpoint produced by each port against its corresponding instruct base, reporting all three averages plus MMLU-Pro alongside the per-test lift.

Three cross-architecture observations are worth noting:

- **Recipe transfer scales with base-model size, with one notable exception.** The 14B and 32B dense Qwen ports both show clear SFT lift over their bases (+13.5 and +13.3 on Avg CTI Test respectively), and the Llama-3.1-8B port also lifts well (+7.3). The Qwen3-MoE port shows a smaller lift (+6.0), and the Foundation-Sec-8B port barely moves (+0.4)—its base is already a security-specialized instruct release, so most of the gain Glaukopis would otherwise deliver is already present in the starting point. At the 8B scale generally, the v21 recipe is not the right shape; the next-vintage research direction includes recipe variants tuned specifically for that parameter range.
- **The Qwen3-MoE port leads on MMLU-Pro among SFT'd checkpoints, but is not the recommended ship.** `athena-cti-sft-qwen3-30b-a3b-thinking-2507-v21-cse` reaches Avg CTI Bench of 63.4 (within 3 points of the dense-32B ship) and retains MMLU-Pro at 53.0—4.4 points below the dense-32B ship's MMLU-Pro of 57.4 but the highest MMLU-Pro among any SFT'd Glaukopis checkpoint that is not the bare 32B. However, MMLU-Pro is a general-intelligence reference and is not part of the CTI use case we ultimately care about (Section 6.1). On the CTI averages that do matter, the dense Qwen-2.5-32B port wins; it is also substantially less expensive to serve and benchmark than the MoE port (the MoE's 128-expert top-8 routing is more expensive both per token and end-to-end through the evaluation harness). The dense Qwen-2.5-32B port is therefore the most efficient and most CTI-capable production ship for the use cases at hand.
- **MMLU-Pro regression varies by port and is largest at the smaller scales.** The dense 32B port loses 11.6 points of MMLU-Pro, the 14B port loses 13.9, the MoE port loses 24.6, and the 8B ports lose 17.6–20.6. At the 32B Qwen scale the regression is the most contained in absolute terms, which is part of why that port is the recommended ship.

Cost Analysis (Draft)

Status.

A per-evaluation cost analysis is being finalized alongside this manuscript. The cost-tracking instrumentation captures, for every model and every benchmark suite, (i) input and output token counts and the relevant per-1K-token rate card for hosted-API checkpoints, and (ii) wall-clock seconds and a per-GPU-hour rate ($2.50/GPU-hour on the 2×H100 reference host) for locally-served vLLM checkpoints. These are blended into a single cost-per-full-suite-evaluation figure that lets accuracy be reported alongside cost as a first-class metric.

Preliminary observations.

The draft cost figures support the broad shape of the argument advanced in the Introduction: on a full-suite evaluation pass (AthenaBench + CyberSOCEval + CyberMetric 2K+10K + MMLU-Pro), the leading frontier proprietary models cost between approximately $160 and $1,150, the strongest open-weights frontier models cost between approximately $11 and $130, and the locally-served Glaukopis v21 32B ship checkpoint costs approximately $3 at the reference 2×H100 hourly rate. The cost ratio between the v21 ship and the leading proprietary frontier is therefore between two and three orders of magnitude per evaluation pass at the time of writing, with the v21 ship landing within a few points of the frontier on Avg CTI Benchmark Score. A finalized cost table with model-by-model break-downs and standardized denominators (per-prompt, per-token, per-evaluation-pass) will appear in this section in the camera-ready version.

Why the cost ratio matters.

Two reasons, both already developed in the Introduction. First, regulated SOC environments have hard limits on what can be sent to a third-party API; locally-served Glaukopis 32B is a deployment-feasible artifact in those environments while the frontier models are not. Second, even where the legal posture allows API use, the per-evaluation cost ratio compounds across analyst workloads—a SOC running thousands of CTI lookups per day cannot afford the frontier pricing the leaderboard reflects. A 32B local checkpoint that closes most of the gap therefore changes the practical deployment calculus, even when the absolute capability gap to the frontier is real.

Open Challenges and Research Directions

Several research directions follow directly from the v21 results and the limitations identified in earlier sections:

- **Combining structured SFT with reinforcement learning from verifiable CTI rewards.** The Glaukopis pipeline is, in its current form, a pure structured-SFT pipeline: every training row is an (instruction, input, output) triple whose loss is computed against the graph-derived ground-truth output, with no explicit reward signal beyond next-token cross-entropy. A natural extension is to follow the SFT chain with a reinforcement-learning phase that scores candidate completions against verifiable CTI signals—e.g., exact-match against the Ariadne-derived gold answer for closed-form tasks (RCM, VSP), or rubric-based scoring for open-ended ones (RMS rationales, TAA explanations). **Minerva**, a recent preprint by Alam et al. Alam et al. [2026] (a co-author of this paper), demonstrates that reinforcement learning with verifiable CTI rewards can deliver measurable additional gains on AthenaBench axes that pure SFT leaves on the table. Similar techniques are noted by the IBM CyberPal research team in the SecKnowledge 2.0 line, which uses RL-style refinement on top of expert-curated instruction data to improve cyber reasoning Levi et al. [2025]. Layering a Minerva-style RL phase on top of the v21 structured-finetuning chain—essentially Stage 5: **Verify**—is the most direct way to extend the Glaukopis pipeline within the existing framework, and is on the roadmap for the next vintage.
- **Larger base models to mitigate general-knowledge erosion.** The general-knowledge erosion documented in Section 6.2 is parameter-count-graded across our dense ports: 8B ports lose 17.6–20.6 MMLU-Pro points, the 14B port loses 13.9, and the 32B port loses 11.6. The trend points to base-model scale-up—e.g., 70–72B—as a mitigation rather than a recipe change: the same structured-finetuning chain run on a larger base is likely to retain a substantially larger fraction of pretrained general capability while still absorbing the CTI corpus, simply because the additional parameters provide more capacity that the SFT pass does not need to overwrite. The trade-off is operational: 70–72B inference is approximately 2× the serving footprint of the current dense 32B ship, and the larger base would need its own port and benchmark sweep. This scale-up is a candidate research direction for a future vintage, and would also be the natural base for a Minerva-style RL extension where retaining out-of-domain reasoning capacity during RL refinement is itself a known concern.
- **Data leakage and benchmark freshness.** Dynamic benchmarks like AthenaBench reduce, but do not eliminate, the risk that training data (e.g., from the same CTI sources) overlaps heavily with evaluation samples. Careful time-based splits and provenance tracking between the Sophia template generator and the benchmarks will be crucial Alam et al. [2025], Cardei [2025].
- **Balancing symbolic templates and LLM creativity.** Template-based generation offers precision and coverage, but LLM-generated paraphrases can increase linguistic diversity and realism (e.g., multiple phrasings of the same CTI scenario). Hybrid pipelines that use Sophia templates to define semantics and LLMs to diversify surface form—while preserving correctness—are a promising area.
- **Incorporating chain-of-thought and explanations.** SecKnowledge 2.0 shows that CoT rationales significantly improve cyber reasoning Levi et al. [2025]. Extending Sophia templates to generate structured reasoning steps (via explicit graph-traversal reasoning) and then augmenting them with LLM-generated natural-language CoT could yield explainable CTI SLMs.
- **Operational evaluation dimensions.** AthenaBench focuses on knowledge and reasoning quality. Security operations also care about robustness to prompt injection, misuse, adversarial CTI, latency, and integration with SIEM/SOAR systems. Benchmarks like SecEval and CyberBench partly address this; extending AthenaBench and complementary suites to cover these aspects remains an important direction Hu et al. [2024], Liu et al. [2024], Alam et al. [2025].
- **Multilingual and cross-regional CTI.** SEvenLLM emphasizes bilingual corpora Ji et al. [2024]. Current CTI benchmarks primarily use English sources, omitting intelligence from other languages that often contain region-specific threats. Sophia templates over multilingual CTI graphs and multilingual benchmark tasks could help close this gap.
- **Human-in-the-loop and analyst trust.** For high-stakes CTI decisions, analysts need to understand why the model suggested a mapping or mitigation and how it connects to CTI sources. Sophia-generated rows that carry their underlying graph bindings, combined with CoT rationales, may enable verifiable, inspectable explanations.

Conclusion

This paper has introduced *structured finetuning* as a discipline imposed on instruction finetuning—every training row computed by a typed query against an authoritative knowledge graph, and the SFT pass decomposed into a chain of stages with distinct dataset shards and hyperparameter regimes—and **Glaukopis** as a working system that instantiates this discipline for cyber threat intelligence. The v21 vintage's empirical evaluation against AthenaBench, CyberSOCEval, CyberMetric, and MMLU-Pro yields three central findings:

- **Structured finetuning delivers substantial CTI lift over a strong open base.** The v21 ship checkpoint adds 7.3 points on Avg CTI Benchmark Score and 13.3 points on Avg CTI Test Score over Qwen-2.5-32B-Instruct (59.0 → 66.3 and 49.6 → 62.9 respectively) without changing parameter count or runtime footprint, and the largest single-axis lift (AthenaBench combined, +22.3 absolute) lands precisely on the benchmark family the chain is designed around.

- **A 32B structured-finetuned model can approach frontier-class CTI performance at substantially lower per-evaluation cost.** The v21 ship sits within 4–7 average-CTI-benchmark points of the leading proprietary and open-weights frontier models—DeepSeek-V4-Pro, Gemini-3-Flash-Preview, GPT-5.5—at approximately two orders of magnitude lower per-evaluation cost. For regulated SOC environments where third-party API use is restricted, this changes the deployment calculus from "frontier or nothing" to "frontier when allowed, on-premises 32B otherwise."
- **The structured-finetuning pipeline is auditable and incrementally refreshable.** Every Glaukopis training row is materialized by Sophia from a typed Cypher query against Ariadne, and every Cypher result is traceable to specific node and edge identifiers that themselves resolve to public-source natural keys. Each SFT stage pushes an intermediate checkpoint and consumes a specific dataset shard, so per-axis regressions are attributable to specific stages and addressable by re-running only the affected ones. The v21 vintage's strict-reproducibility experiment over v18.1 additionally isolated data-build non-determinism in the template-generation substrate from recipe-level drift, informing the seed-pinning research direction for the next vintage.

The result is a principled and reproducible pathway from authoritative public CTI sources (via Ariadne) through graph-template-driven instruction generation (Sophia) to a chained SFT recipe that produces deployable cyber security expert small language models grounded simultaneously in CTI knowledge bases, instruction-finetuning theory, and a rigorous benchmark portfolio. Glaukopis is one instantiation of that pathway; the broader argument it supports is that, for CTI workflows, on-premises 32B-scale models trained under disciplined structured finetuning are a viable alternative to frontier proprietary models in deployment scenarios where the latter cannot, in fact, be deployed.

Two follow-on directions appear most likely to move the leaderboard further, both discussed at greater length in Section 7. The first is layering a reinforcement-learning phase with verifiable CTI rewards on top of the structured-finetuning chain, an approach demonstrated to be effective for CTI reasoning in the Minerva work by Alam et al. Alam et al. [2026] and used in related form by IBM's CyberPal research line on SecKnowledge 2.0 Levi et al. [2025]. The second is moving the base model to the 70–72B scale, where the parameter-count-graded general-knowledge erosion documented in Section 6.2 is expected to be substantially smaller and where the additional capacity gives an RL refinement phase more room to operate without re-perturbing the CTI gains accumulated during SFT.

Conflicts of interest

None declared.

Funding

No external funding supported this manuscript.

Data availability

The Ariadne CTI knowledge graph is built from twelve public CTI sources listed in Section 4.3; each is independently accessible from its original publisher and is downloaded and parsed by the open populator script described therein. The v21 template manifests, the seven IFT dataset shards, the per-stage SFT launchers, and the evaluation-harness wrappers used to produce the results in Section 6 are maintained internally at Athena Security Group; release plans are under discussion. Cumulative HF checkpoints (`asg-ai/athena-cti-sft-qwen25-32b-v21-{core,taa,cse,recal-32b}` and the cross-architecture ports) are available on the Hugging Face Hub.

Acknowledgments

The authors thank colleagues at Florida Atlantic University's Department of Electrical Engineering and Computer Science and at Athena Security Group for discussions that informed this work, and especially Dr. Ionut Cardei and Mamoon Khan for the design and implementation of the Sophia template generator that produces every Glaukopis training row.

References

Common attack pattern enumeration and classification (CAPEC). `https://capec.mitre.org/`, a. Accessed 2025-12-03.

Common vulnerabilities and exposures (CVE). `https://www.cve.org/`, b. Accessed 2025-12-03.

Common weakness enumeration (CWE). `https://cwe.mitre.org/`, c. Accessed 2025-12-03.

M. T. Alam, D. Bhusal, N. Shah, and N. Rastogi. Ctibench: A benchmark for evaluating LLMs in cyber threat intelligence. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024. URL `https://arxiv.org/abs/2406.07508`.

M. T. Alam, D. Bhusal, S. Ahmad, N. Rastogi, and P. J. Worth. Athenabench: A dynamic benchmark for evaluating LLMs in cyber threat intelligence. *arXiv preprint arXiv:2511.01144*, 2025. URL `https://arxiv.org/abs/2511.01144`.

M. T. Alam et al. Minerva: Reinforcement learning for cyber threat intelligence reasoning. *arXiv preprint*, 2026. Preprint; exact arXiv identifier and author list to be confirmed before submission.

M. Bhatt, S. Chennabasappa, C. Nikolaidis, S. Wan, I. Evtimov, D. Gabi, D. Song, F. Ahmad, C. Aschermann, L. Fontana, S. Frolov, R. P. Giri, D. Kapil, Y. Kozyrakis, D. LeBlanc, J. Milazzo, A. Straumann, G. Synnaeve, V. Vontimitta, S. Whitman, and J. Saxe. Purple Llama CyberSecEval: A secure coding benchmark for language models. *arXiv preprint arXiv:2312.04724*, 2023. URL `https://arxiv.org/abs/2312.04724`.

M. Bhatt, S. Chennabasappa, Y. Li, C. Nikolaidis, D. Song, S. Wan, F. Ahmad, C. Aschermann, Y. Chen, D. Kapil, D. Molnar, S. Whitman, and J. Saxe. CyberSecEval 2: A wide-ranging cybersecurity evaluation suite for large language models. *arXiv preprint arXiv:2404.13161*, 2024. URL `https://arxiv.org/abs/2404.13161`.

I. Cardei. Instruction fine tuning dataset design. Technical report, Athena Labs, Athena Security Group, 2025. Internal design

document describing the Athena CTI Neo4j graph schema and template-based IFT dataset generation.

H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus, E. Li, X. Wang, M. Dehghani, S. Brahma, et al. Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*, 2022. URL https://arxiv.org/abs/2210.11416.

C. Fieblinger, A. M. Tjoa, et al. Actionable cyber threat intelligence using knowledge graphs and large language models. *arXiv preprint*, 2024.

S. Gururangan, A. Marasović, S. Swayamdipta, K. Lo, I. Beltagy, D. Downey, and N. A. Smith. Don't stop pretraining: Adapt language models to domains and tasks. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 8342–8360, 2020. URL https://arxiv.org/abs/2004.10964.

H. Hu, P. Zhang, et al. Seceval: A comprehensive benchmark for evaluating cybersecurity knowledge in foundation models. *arXiv preprint*, 2024.

H. Ji, J. Yang, L. Chai, C. Wei, L. Yang, Y. Duan, Y. Wang, T. Sun, H. Guo, T. Li, et al. Sevenllm: Benchmarking, eliciting, and enhancing abilities of large language models in cyber threat intelligence. *arXiv preprint arXiv:2405.03446*, 2024. URL https://arxiv.org/abs/2405.03446.

M. Levi, Y. Allouche, D. Ohayon, and A. Puzanov. Cyberpal.ai: Empowering LLMs with expert-driven cybersecurity instructions. *arXiv preprint arXiv:2408.09304*, 2024. URL https://arxiv.org/abs/2408.09304.

M. Levi, D. Ohayon, A. Blobstein, R. Sagi, I. Molloy, and Y. Allouche. Toward cybersecurity-expert small language models. *arXiv preprint arXiv:2510.14113*, 2025. URL https://arxiv.org/abs/2510.14113.

N. Li, A. Pan, A. Gopal, S. Yue, D. Berrios, A. Gatti, J. D. Li, A.-K. Dombrowski, S. Goel, L. Phan, G. Mukobi, N. Helm-Burger, R. Lababidi, L. Justen, A. B. Liu, M. Chen, I. Barrass, O. Zhang, X. Zhu, R. Tamirisa, B. Bharathi, A. Khoja, Z. Zhao, A. Herbert-Voss, C. B. Breuer, S. Marks, O. Patel, A. Zou, M. Mazeika, Z. Wang, P. Oswal, W. Lin, A. A. Hunt, J. Tienken-Harder, K. Y. Shih, K. Talley, J. Guan, R. Kaplan, I. Steneker, D. Campbell, B. Jokubaitis, A. Levinson, J. Wang, W. Qian, K. K. Karmakar, S. Basart, S. Fitz, M. Levine, P. Kumaraguru, U. Tupakula, V. Varadharajan, R. Wang, Y. Shoshitaishvili, J. Ba, K. M. Esvelt, A. Wang, and D. Hendrycks. The WMDP benchmark: Measuring and reducing malicious use with unlearning. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. URL https://arxiv.org/abs/2403.03218.

Z. Liu, J. Shi, and J. F. Buford. Cyberbench: A multi-task benchmark for evaluating large language models in cybersecurity. *arXiv preprint arXiv:2407.08665*, 2024. URL https://arxiv.org/abs/2407.08665.

R. McMillan. Defining threat intelligence. Technical report, Gartner, 2013.

Meta AI / Purple Llama Team. CyberSOCEval: Benchmarking LLMs for malware analysis and threat intelligence reasoning. https://github.com/meta-llama/PurpleLlama, 2025. arXiv preprint; exact citation form to be confirmed.

L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*, 2022. URL https://arxiv.org/abs/2203.02155.

B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas. MITRE ATT&CK: Design and philosophy. Technical Report MP180360, The MITRE Corporation, 2018. URL https://attack.mitre.org/resources/.

N. Tihanyi, M. A. Ferrag, R. Jain, T. Bisztray, and M. Debbah. CyberMetric: A benchmark dataset based on retrieval-augmented generation for evaluating LLMs in cybersecurity knowledge. *arXiv preprint arXiv:2402.07688*, 2024. URL https://arxiv.org/abs/2402.07688.

J. Wei, M. Bosma, V. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations (ICLR)*, 2022a. URL https://arxiv.org/abs/2109.01652.

J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*, 2022b. URL https://arxiv.org/abs/2201.11903.

X. Wu, L. Wang, et al. Open-cykg: Constructing a cyber threat intelligence knowledge graph from open sources. *arXiv preprint*, 2022.

Y. Yu, J. Kang, et al. Ctikg: A large-scale, high-quality cyber threat intelligence knowledge graph. *arXiv preprint*, 2024.

Y. Zhang, H. Chen, et al. Cyberkg: A cyber threat intelligence knowledge graph for threat analysis. *arXiv preprint*, 2023.

Z. Zhao, H. Li, et al. K-ctiaa: Knowledge graph enhanced cyber threat intelligence action analysis. *arXiv preprint*, 2024.